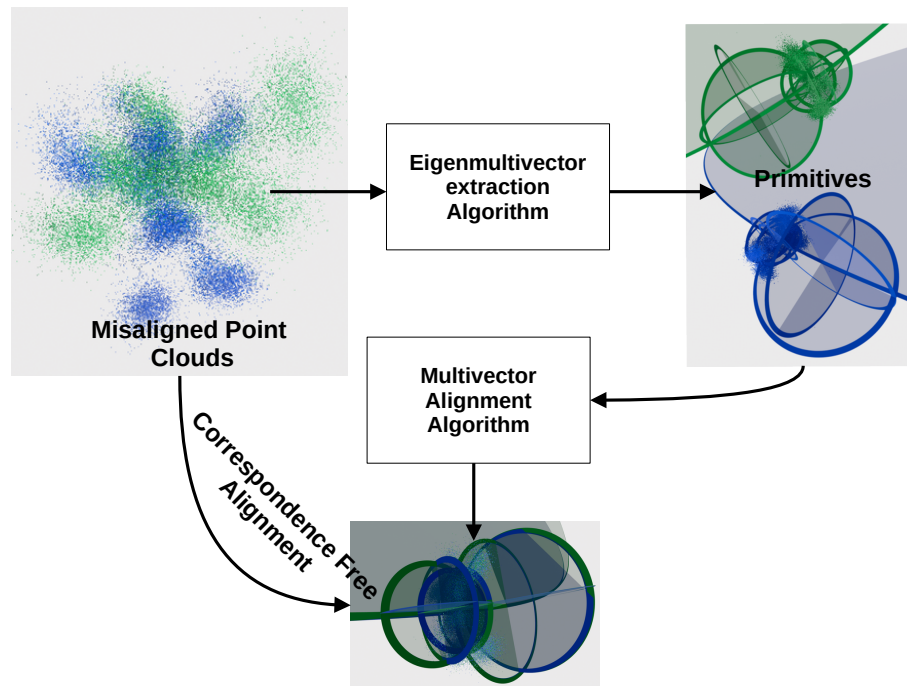


Graphical Abstract

Multivector-Based Cloud Alignment without Correspondences



Highlights

Multivector-Based Cloud Alignment without Correspondences

- Introduces a correspondence-free cloud alignment method using conformal geometric algebra
- Formulates alignment as an eigenmultivector problem avoiding pairwise correspondences
- Derives translation- and rotation-equivariant eigenmultivector properties in CGA
- Proposes a multivector coefficients algorithm to estimate rigid transformations numerically
- Demonstrates robustness to noise and comparable accuracy to state-of-the-art alignment methods

Multivector-Based Cloud Alignment without Correspondences

Abstract

We present, for the first time, a novel theoretical approach to address the problem of correspondence free multivector cloud alignment in conformal geometric algebra. Such formalism achieves several favorable properties. Primarily, it forms an orthogonal automorphism that extends beyond the typical vector space to the entire conformal geometric algebra while respecting the multivector grading. In essence, the alignment corresponds to an orthogonal transformation (i.e., scale, translation, and rotation) that belongs to the $SO(4, 1)$ group in conformal geometric algebra. We will show that such formalism is able to: (i) perform the alignment without directly accessing the input multivectors. Instead, we use primitives or geometric objects provided by the conformal model - the multivectors, (ii) the geometric objects are obtained by solving a multilinear eigenvalue problem to find sets of eigenmultivectors. In this way, we can explicitly avoid solving the correspondences in the alignment process. Most importantly, this offers rotation and translation equivariant properties between the input multivectors and the eigenmultivectors. Experimental evaluation is conducted in datasets commonly used in point cloud alignment, to testify the usefulness of the approach with emphasis to ambiguities arising from high levels of noise. For code access, please refer to [Registration-GA](#).

Keywords: Conformal Geometric Algebra, Point Cloud alignment, Multivector Cloud alignment, Computer Vision, Correspondence Free alignment

1. Introduction

Point cloud alignment constitutes a central topic in computer vision, recognized as both essential and challenging. Its applications span a wide range of areas, including 3D scene reconstruction [Agarwal et al. \(2011\)](#); [Schonberger and Frahm \(2016\)](#), object pose estimation [Dang et al. \(2022\)](#); [Wong et al. \(2017\)](#), and LiDAR-based SLAM [Deschaud \(2018\)](#); [Zhang and Singh \(2014\)](#); [Lu et al. \(2021\)](#). Essentially, the task consists of registering two point clouds that partially coincide by determining the 3D rotational and translational transformation connecting them

A prevalent method for handling large-scale alignment problems is a two-stage framework. The first stage involves generating point descriptors [Choy et al. \(2019\)](#); [Deng et al. \(2018\)](#); [Frome et al. \(2004\)](#); [Rusu et al. \(2009\)](#); [Salti et al. \(2014\)](#); [Zeng et al. \(2017\)](#), followed by a correspondence step that matches points of the clouds to infer the transformation geometrically. Designing reliable descriptors is therefore a key factor in enhancing the overall robustness of this pipeline. Considerable research has explored both classical and deep learning-based descriptor techniques [Bai et al. \(2020\)](#); [Huang et al. \(2021\)](#); [Wang et al. \(2022\)](#).

Nonetheless, the obtained point correspondences are often prone to errors, especially in difficult scenarios, for instance, low overlap, recurring patterns, or noisy data, which can adversely affect alignment quality. To mitigate these issues, numerous outlier removal methodologies are available in the literature to filter incorrect matches, for instance random sample consensus (RANSAC) [Fischler and Bolles \(1981\)](#), point-wise descriptor similarity measures [Fischler and Bolles \(1981\)](#); [Lowe \(2004\)](#), and group-based spatial consistency checks [Yang et al. \(2019\)](#).

In this paper we discuss a hitherto untouched aspect of conformal geometric algebra (CGA) - its application to *point cloud alignment without correspondences*. CGA is emerging as a new approach to address computer vision problems [Pillai and Megalingam \(2020\)](#); [Ruhe et al. \(2023a\)](#), having the advantage of offering a simple yet efficient representation of geometric objects and transformations. Particularly, we design a new CGA algorithm that is able to determine rigid transformations between general multivectors. Our contribution is twofold: (i) we introduce a novel CGA algorithm that estimates rigid transformations between two multivector clouds, and (ii) a correspondence free algorithm that solves the alignment by using CGA. The correspondence free algorithm comprises the two following steps: (i) first, we solve an eigenvalue problem for each multivector cloud, (ii) we then use the eigenmultivectors of each multivector cloud to estimate the rigid transformation. Finally, it will be shown that the proposed algorithm does not impose any constraints regarding the multivectors that are being registered. To summarize, our contributions comprise the following novel algorithms:

1. An algorithm for extracting eigenmultivectors that is equivariant to both 3D rotations and translations. It is general since it can be used for any type of objects in CGA (Sec. 3.2).
2. An algorithm, using a multivector coefficients approach, to estimate the rotation and translation between objects in CGA, being the objects of any type *e.g.*, mixed grade multivectors (Sec. 3.3).

The following key advantages characterize the distinctiveness of our approach: (i) no need to define a cost function for the multivector clouds (this is specially

important for pseudo-Euclidean spaces where norms do not represent the usual notion of a ‘physical quantity’), (ii) nor to find multivector correspondences between multivector clouds, (iii) it is robust to high levels of noise, (iv) the eigenmultivectors act as the principal components of the multivector cloud, which best explains the data statistics (which are $SO(p, q)$ equivariant to the multivector clouds), and (v) the eigenvalues provide $SO(p, q)$ invariant quantities.

2. Related work

Traditional class of approaches can be framed in PCA based statistical methods [Celik and Ma \(2008\)](#); [Gonzalez \(2009\)](#); [Rehman and Lee \(2018\)](#) or *purely geometric* methods [Bustos and Chin \(2017\)](#); [Chen et al. \(2022\)](#); [Leordeanu and Hebert \(2005\)](#); [Yang et al. \(2020b\)](#); [Zhou et al. \(2016\)](#). However, with the advances of deep learning in the 3D vision field, other classes have rapidly emerged. *End-to-end alignment methods* have achieved increasing attention. One of the pioneering works is the DCP [Wang and Solomon \(2019\)](#) that exploits feature similarity to establish pseudo correspondences for SVD-based transformation estimation. But other end-to-end methods, *e.g.* [Choy et al. \(2020\)](#); [Fu et al. \(2021\)](#); [Li et al. \(2020b, 2019, 2020c\)](#); [Qin et al. \(2022\)](#); [Zhu and Fang \(2020\)](#), contributed for the relevance of this class of approaches. Other class adopts *learning-based* strategy where feature descriptors for 3D matching are exploited constituting an advance when compared to the hand-crafted descriptors, *e.g.* [Frome et al. \(2004\)](#); [Rusu et al. \(2009\)](#); [Salti et al. \(2014\)](#). 3DMArch [Zeng et al. \(2017\)](#) is known as one of the representative learning-based methods using Siamese 3D CNN to learn the local geometric feature via contrastive loss. Other approaches are also available, exploring fully convolutional network for dense feature extraction [Choy et al. \(2019\)](#); [Bai et al. \(2020\)](#); [Huang et al. \(2021\)](#); [Wang et al. \(2022\)](#); [Qin et al. \(2022\)](#); [Li and Harada \(2022\)](#). Despite significant progress in deep-based feature descriptor, generating mismatched correspondences (outliers) remains unavoidable. Outlier rejection thus becomes as a natural step to be accounted for, often using RANSAC [Fischler and Bolles \(1981\)](#) or its variants [Barath and Matas \(2018\)](#); [Le et al. \(2019\)](#); [Li et al. \(2020a\)](#) that use repeated sampling and verification for outlier rejection. However, these methods tend to have a high time cost, particularly in scenes with a significant outlier ratio. *Correspondences-free alignment* methods, are however, able to surpass the above rejection step as they directly estimate the rigid transformation, usually achieved by establishing an end-to-end differentiable network. This can be accomplished by either using *soft correspondence* methods [Fu et al. \(2021\)](#); [Lu et al. \(2021, 2019\)](#); [Wang and Solomon \(2019\)](#); [Yew and Lee \(2020\)](#) or *direct regression* methods. The latter class of methods was pioneered by [Aoki et al. \(2019\)](#) that uses PointNet [Qi et al. \(2017\)](#) as to extract global features and introducing

differentiable Lucas-Kanade algorithm [Lucas and Kanade \(1981\)](#) to minimize the feature distance. Several other works [Huang et al. \(2020\)](#); [Li et al. \(2021\)](#); [Yuan et al. \(2020\)](#) followed this line of research. Although encouraging results have been reached, some difficulties still persist in both classes of methodologies. Concretely, soft correspondence struggles to generalize to unseen setups with several sensors, and the direct regression methods still face difficulties in large-scale scenes.

This paper proposes a different strategy, where the alignment is viewed as a *geometric transformation* of the input data that is performed using multivectors in CGA. Indeed, CGA [Clifford \(1871\)](#); [Hestenes \(2015\)](#); [Dorst and Mann \(2002\)](#); [Artin \(2016\)](#) has several favorable properties. CGA can easily avoid the PCA problem of 180° when computing the eigenvectors of the covariance matrix to estimate the rotation [Rehman and Lee \(2018\)](#). Also, (i) it efficiently encodes the transformations and the invariant elements of classic geometries, (ii) it is intuitive to manipulate geometric objects and treat them as operators, meaning that, we can easily define transformations via the geometric product of objects, being the objects vectors, points, lines, and planes, (iii) it generalizes over dimensions in the sense that transformations and objects are built independently of the space dimensionality, and (iv) it allows to unify a large of mathematical systems like vector algebra, projective geometry, quaternions and Plücker coordinates. CGA is an emerging topic in computer vision and some works exist. In [Ruhe et al. \(2023b\)](#) GCANs is proposed as a novel method to incorporate geometry-guided transformations into neural networks. CGAPoseNet+GCAN [Pepe et al. \(2024\)](#) enhances CGAPoseNet [Pepe and Lasenby \(2023\)](#) using Clifford Geometric Algebra to unify quaternions and translation vectors into a single mathematical object, the motor, which can be used to uniquely describe camera poses.

In this paper, we develop a novel alignment framework using CGA, with the following structure. In Sec. 3 we specifically address our contributions. Sec. 4 thoroughly validates our proposal. The conclusions are addressed in Sec. 5. In the Appendix we provide all the proofs of the formulated theorems and review the quintessence of modern Geometric Algebra.

3. Background and Motivation

This section outlines our contributions for multivector cloud alignment using CGA and the exposition is as follows. In Sec. 3.2, the *eigenmultivector extraction* approach is proposed to extract eigenmultivectors from multivector clouds that will be used to estimate the rigid transformation. We notice that our proposal deals with any type of multivectors. The rationale behind the proposal is that it takes as the input each multivector cloud and solves an eigenvalue problem for each cloud, retrieving unique eigenmultivectors, forming a basis for the CGA. In the case where

Table 1: Difference between Vanilla Geometric Algebra and Conformal Geometric Algebra when performing alignment. The vectors $\mathbf{p}_i$ are eigenvectors of f with corresponding real eigenvalues λ_i (left). The multivectors $\mathbf{P}_i$ are eigenmultivectors of F with corresponding real eigenvalues ν_i (right). The $\mathbf{P}^j$'s are the reciprocal multivectors such that $\mathbf{P}_i * \mathbf{P}^j = \delta_{ij}$ (* is the scalar product operator). The vectors $\mathbf{z}, \mathbf{x}_i \in \mathcal{A}_3$ and the multivectors $\mathbf{Z}, \mathbf{X}_i \in \mathcal{G}_{4,1}$.

VANILLA GEOMETRIC ALGEBRA	CONFORMAL GEOMETRIC ALGEBRA
$f(\mathbf{z}) = \sum_i \mathbf{z} \cdot \mathbf{x}_i \mathbf{x}_i = \sum_i \lambda_i \mathbf{z} \cdot \mathbf{p}_i \mathbf{p}_i^{-1}$ $\Downarrow \mathbf{x}_i \rightarrow R(\mathbf{x}_i) \Downarrow$ $g(\mathbf{z}) = \sum_i \mathbf{z} \cdot R(\mathbf{x}_i) R(\mathbf{x}_i)$ $= \sum_i \lambda_i \mathbf{z} \cdot R(\mathbf{p}_i) R(\mathbf{p}_i^{-1})$	$F(\mathbf{Z}) = \sum_i \mathbf{X}_i \mathbf{Z} \mathbf{X}_i = \sum_i \nu_i \mathbf{Z} * \mathbf{P}^i \mathbf{P}_i$ $\Downarrow \mathbf{X}_i \rightarrow \underline{T}R(\mathbf{X}_i) \Downarrow$ $G(\mathbf{Z}) = \sum_i \underline{T}R(\mathbf{X}_i) \mathbf{Z} \underline{T}R(\mathbf{X}_i)$ $= \sum_i \nu_i \mathbf{Z} * \underline{T}R(\mathbf{P}^i) \underline{T}R(\mathbf{P}_i)$

the eigenmultivectors are of unique grade, they can be interpreted as primitives objects such as spheres, point pairs, and circles. Otherwise, only form a basis for the entire conformal geometric algebra, not necessarily being blades. The obtained eigenmultivectors from Sec. 3.2 are then used by the proposed *coefficients* method detailed in Sec. 3.3, that will estimate the translations and rotation. The coefficients method deals with multivectors which are, at least, bivector plus trivector. We provide solutions with and without correspondences assumption.

3.1. Why CGA for point cloud alignment

While traditional approaches, *e.g.* PCA based methods, only offer rotation equivariant and invariant properties through the decomposition of a covariance matrix, with CGA, the eigenvalues and eigenmultivectors provide both rotation and translation invariant and equivariant properties. This offers an attractive property when dealing with point cloud alignment. In Tab. 1 (left), we show how the decomposition of the covariance matrix (*i.e.* the linear function f) is affected by a rotation. In Tab. 1 (right), we show how the decomposition of a multilinear function F is affected by a rotation and a translation. Note that the eigenvalues of f are only unaffected by the rotation, while the eigenvalues of F remain unchanged with both the translation and rotation.

3.2. Eigenmultivector Extraction

In this section, we describe a novel methodology that takes as the inputs the multivectors from each cloud, and retrieves the corresponding eigenmultivectors.

We further show that, under noise-free assumptions, the eigenmultivectors of two multivector clouds are related via rotation and translation.

Formally, let us consider two multivector clouds $X_i \in \mathcal{G}_{4,1}$ and $Y_i \in \mathcal{G}_{4,1}$, for $i = 1, 2, \dots, \ell$, where $\mathcal{G}_{p,q}$ denotes the geometric algebra of a 2^{p+q} dimensional real linear vector space. We aim to find the rotator $R \in \mathcal{G}_3^+$, such that $RR^\dagger = 1$, and a translation vector $t \in \mathcal{A}_3$ which best aligns the multivector clouds. Here, $\mathcal{G}_3^+$ denotes the even subalgebra of $\mathcal{G}_3$ and $\mathcal{A}_{p,q}$ is a $(p+q)$ -dimensional vector space over the field of real numbers $\mathbb{R}$. We start by assuming that

$$Y_i = \underline{T}R(X_{j_i}) + N_i \quad (3.1)$$

where N_i is zero mean Gaussian noise, and where we define $T(z) \equiv TzT^\dagger$, $R(z) \equiv RzR^\dagger$ for $z \in \mathcal{A}_{4,1}$, with $T = 1 + e_\infty t/2$. In (3.1), the notation j_i stands for the unique assignment $X_{j_i} \leftrightarrow Y_i$.

The notation $\underline{T}, \underline{R}$ (used in (3.1)) and $\bar{T}, \bar{R}$, are used to denote the differential and the adjoint outermorphisms of a linear function T, R , respectively (for a detailed description of linear transformation and outermorphisms see [Hestenes and Sobczyk \(1984\)](#)). Our proposal starts by defining the following multilinear functions for each of the multivector clouds

$$F(Z) \equiv \sum_{i=1}^{\ell} X_i Z X_i, \quad G(Z) \equiv \sum_{i=1}^{\ell} Y_i Z Y_i \quad (3.2a, 3.2b)$$

Given the equalities in (3.1) and (3.2) a relationship between F and G is given by

$$G(Z) = \underline{U}F\bar{U}(Z) + N(Z) \quad (3.3a)$$

where $\underline{U}$ is the differential outermorphism of the linear function

$$U(z) \equiv UzU^\dagger = TRzR^\dagger T^\dagger = TR(z) \quad (3.3b)$$

where $z \in \mathcal{A}_{4,1}$ and with the motor $U \equiv TR$. The differential outermorphism $\underline{U}$ of U , extends through the multivector space $\mathcal{G}_{4,1}$ and takes the form $\underline{U}(Z) = UZU^\dagger$, with $Z \in \mathcal{G}_{4,1}$. N is the noisy multilinear function given by

$$N(Z) = \sum_{i=1}^{\ell} N_i Z N_i + N_i Z X_i + X_i Z N_i \quad (3.3c)$$

Notice that the relation (3.3a) is invariant under any permutation of the points Y_{j_i} . An eigenmultivector can be viewed as the solution to an eigenvalue problem $F(Z) = \lambda Z$ in the space of multivectors. Specifically, in the context of multilinear

transformations the concept of eigenvector can be extended to eigenmultivectors (see *Definition 6* and [D](#), for a detailed description of Eigenmultivectors).

In the following theorem, we explain how the eigenmultivectors of F and G are related. These eigenmultivectors are equivariant with respect to the multivector clouds X_i and Y_i , for $i = 1, 2, \dots, \ell$, and used to estimate orthogonal transformations.

Theorem 3.1. *Let $P_1, P_2, \dots, P_m \in \mathcal{G}_{4,1}$ be unique eigenmultivectors of F , and let $Q_1, Q_2, \dots, Q_m \in \mathcal{G}_{4,1}$ be unique eigenmultivectors of G . Assume that (3.1) is noise-free. Then, the eigenvectors of F and G are related via a scaled rigid transformation. That is, the following equality holds*

$$Q_i = s_i U(P_i) \quad (3.4)$$

where the scalar $s_i \in \mathbb{R}$. Furthermore the eigenvalues of F and G are equal.

The proof follows directly from Lemma [D.1](#) in Appendix [D](#). The eigenmultivectors of the multilinear transformation F have the important property making them equivariant to the input multivector cloud, i.e. $X_1, X_2, \dots, X_\ell$. In particular, when the X_i suffers an orthogonal transformation U , then the eigenmultivectors will also suffer the same transformation (with the scaling ambiguity). Another important property is that if they have an associated unique eigenvalue then by ordering the eigenmultivectors by their eigenvalues the equality (3.4) holds. To estimate the scalar s_i in (3.4) we consider the following *Corollary*.

Corollary 3.1. *Under the assumptions of Theorem 3.1 the scalar in (3.4) can be determined as follows*

$$s_i = \langle Q_i Q_{\text{ref}} \rangle \langle P_i P_{\text{ref}} \rangle^{-1} \quad (3.5)$$

where P_{ref} and Q_{ref} are both multivectors in $\mathcal{G}_{4,1}$ that are related as $Q_{\text{ref}} = U(P_{\text{ref}})$.

See [proof](#) on page 46. In the particular case where X_i and Y_i are grade one multivectors, we can readily provide simple, yet effective quantities which are used to estimate the scalars s_i . Particularly, the multivectors in (3.5) are chosen to be $P_{\text{ref}} = (1 + i)(e_\infty + \bar{X} \wedge e_\infty)$ and $Q_{\text{ref}} = (1 + i)(e_\infty + \bar{Y} \wedge e_\infty)$, with $\bar{X} = \frac{1}{\ell} \sum_{i=1}^{\ell} X_i$ and $\bar{Y} = \frac{1}{\ell} \sum_{i=1}^{\ell} Y_i$ (see *Theorem D.3*). *Theorem 3.1* and *Corollary 3.1* above, suggest that the alignment can be accomplished *without* correspondences as the eigenmultivectors are ordered by their respective (and unique) eigenvalues. The synopsis of *alignment with eigenmultivector extraction* approach is shown in Alg. 1. We notice that for the step 3 of the Alg. 1, we introduce a new algorithm based on *multivector coefficients* that is described in Sec. 3.3. Note that the alignment algorithm computes the normalized eigenmultivectors using Alg. 2, which in turn,

takes as input data multiple multivectors and extracts equivariant multivectors by taking the eigendecomposition of the covariance matrix (the multilinear function F and G) of the multivector clouds.

Algorithm 1 Rigid Transformation Estimation with Eignemultivector Extraction

- 1) **Input Data:** $X_1, X_2, \dots, X_\ell, Y_1, Y_2, \dots, Y_\ell$
 - 2) Use Alg. 2 to compute the normalized eigenmultivectors P_i and Q_i , for $i = 1, \dots, m$ of the multivector clouds X_i and Y_i for $i = 1, \dots, \ell$
 - 3) Take P_i and Q_i to estimate the rigid transformation $\widehat{U} = \widehat{TR}$, using Alg. 3 (Sec. 3.3)
-

3.3. The Multivector Coefficients Approach

In this section we provide a novel algorithm that can be used to estimate a rigid transformation from a set of multivectors. Concretely, we provide a strategy that receives as the input any two multivector clouds in $\mathcal{G}_{4,1}$ and estimates the rigid transformation between them. The method described herein works with any known correspondences, in particular the correspondences provided by the eigenmultivector extraction method introduced in Sec. 3.2.

Algorithm 2 Eigenmultivector Extraction

- 1) **Input Data:** $X_1, X_2, \dots, X_\ell$
 - 2) Define the multilinear function $F(Z) = \sum_{i=1}^{\ell} X_i Z X_i$, (eq. (3.2));
 - 3) Compute the eigenmultivectors $P_1, P_2, \dots, P_m$ and eigenvalues $\lambda_1, \lambda_2, \dots, \lambda_m$ of F (Def. (6))
 - 4) Order the eigenmultivectors by their respective eigenvalues. Let P_{ref} be some equivariant function of the points X_i
 - 5) Scale the eigenmultivectors as $P_i \leftarrow P_i / \langle P_i P_{\text{ref}} \rangle$, (eq. (3.5));
 - 6) **Return** the eigenmultivectors $P_1, P_2, \dots, P_m$
-

Let $P_1, P_2, \dots, P_m$ and $Q_1, Q_2, \dots, Q_m$ be two sets of multivectors that are related via $Q_i = U(P_i) = TR(P_i)$, that is, we assume they relate via a rigid transformations and where the *correspondences are known*. Our approach starts by breaking down the multivectors P_i and Q_i in $\mathcal{G}_{4,1}$ into its constituent parts in $\mathcal{G}_3$. Considering a single multivector $P \in \mathcal{G}_{4,1}$, we can write

$$P \equiv e_o \mathcal{P}_1 + e_\infty \mathcal{P}_2 + e_o \wedge e_\infty \mathcal{P}_3 + \mathcal{P}_4 \quad (3.6)$$

where $\mathcal{P}_1, \mathcal{P}_2, \dots, \mathcal{P}_4 \in G_3$ are the coefficients of the multivector $P \in \mathcal{G}_{4,1}$, and e_o and e_∞ are defined in (C.1). To determine the coefficients from a multivector we define a set of functions $C_i(\cdot)$, for $i = 1, \dots, 4$, which provide a mapping from a multivector P to its coefficients, this mapping is defined in (C.6).

Next, in Sec. 3.3.1 we address how the coefficients $\mathcal{P}_1, \mathcal{P}_2, \dots, \mathcal{P}_4$ of P are affected under a rigid transformations (*Theorem 3.2*). In Sec. 3.3.2, we propose a

methodology to separately estimate the rotation and translation by using *Theorems 3.3* and *3.4*. In summary, we will first estimate the rotation using *Theorem 3.3* followed by the translation using *Theorem 3.4*.

3.3.1. Relation between the coefficients under the rigid transformation

The following theorem states how the coefficients are related if the two multivector clouds are related under a rigid transformation.

Theorem 3.2. *Let the multivectors $\mathbf{P} \in \mathcal{G}_{4,1}$ and $\mathbf{Q} \in \mathcal{G}_{4,1}$ be related via the composition of a translation T and a rotation R , that is, $\mathbf{Q} = \underline{T}\underline{R}(\mathbf{P})$. Then the coefficients $\mathcal{P}_i = C_i(\mathbf{P})$, $\mathcal{Q}_i = C_i(\mathbf{Q})$ are related as*

$$\begin{aligned}\mathcal{Q}_1 &= \underline{R}(\mathcal{P}_1) \\ \mathcal{Q}_2 &= \frac{1}{2}\underline{t}^2\underline{R}(\mathcal{P}_1) - \underline{t} \wedge (\underline{t} \cdot \underline{R}(\mathcal{P}_1)) \\ &\quad + \underline{R}(\mathcal{P}_2) - \underline{t} \wedge \underline{R}(\mathcal{P}_3) + \underline{t} \cdot \underline{R}(\mathcal{P}_4) \\ \mathcal{Q}_3 &= \underline{t} \cdot \underline{R}(\mathcal{P}_1) + \underline{R}(\mathcal{P}_3) \\ \mathcal{Q}_4 &= \underline{t} \wedge \underline{R}(\mathcal{P}_1) + \underline{R}(\mathcal{P}_4)\end{aligned}\tag{3.7a}$$

Furthermore, we can also show how the coefficients are affected by the inverse transformation $\mathbf{P} = \bar{R}\bar{T}(\mathbf{Q})$, having

$$\begin{aligned}\mathcal{P}_1 &= \bar{R}(\mathcal{Q}_1) \\ \mathcal{P}_2 &= \frac{1}{2}\underline{t}^2\bar{R}(\mathcal{Q}_1) + \bar{R}(\underline{t} \wedge (\underline{t} \cdot \mathcal{Q}_1)) \\ &\quad + \bar{R}(\mathcal{Q}_2) + \bar{R}(\underline{t} \wedge \mathcal{Q}_3) - \bar{R}(\underline{t} \cdot \mathcal{Q}_4) \\ \mathcal{P}_3 &= -\bar{R}(\underline{t} \cdot \mathcal{Q}_1) + \bar{R}(\mathcal{Q}_3) \\ \mathcal{P}_4 &= -\bar{R}(\underline{t} \wedge \mathcal{Q}_1) + \bar{R}(\mathcal{Q}_4)\end{aligned}\tag{3.7b}$$

See [proof](#) on page 39. Note that from (C.6) we can also write

$$\mathcal{Q}_i = C_i(\underline{T}\underline{R}(\mathbf{P})), \mathcal{P}_i = C_i(\bar{R}\bar{T}(\mathbf{Q})), \text{ for } i = 1, \dots, 4.\tag{3.8a, 3.8b}$$

3.3.2. Estimating R and t from multivector coefficients

Notice that the first coefficients of $\mathbf{P}$ and $\mathbf{Q}$ relate via a rotation (see first equation of (3.7a)). Now, let $\mathbf{A}_i = C_1(\mathbf{P}_i) \in G_3$ and $\mathbf{B}_i = C_1(\mathbf{Q}_i) \in G_3$. It is straightforward to obtain $\mathbf{B}_i = \underline{R}(\mathbf{A}_i)$. To estimate the rotation between the multivectors $\mathbf{A}_i$ and $\mathbf{B}_i$ we state the following theorem.

Theorem 3.3. Let $A_i, B_i \in \mathcal{G}_3$, with $i = 1, \dots, \ell$, be two sets of ℓ multivectors and assume that they are related by

$$B_i = R A_i R^\dagger + N_i \quad (3.9a)$$

where $RR^\dagger = 1$ and $N_i \in \mathcal{G}_3$ is Gaussian noise. Then, the optimal rotor R is determined by minimizing the following Lagrangian

$$\mathcal{L}(R) = \sum_{i=1}^{\ell} \|B_i - R A_i R^\dagger\|^2 + \lambda \langle RR^\dagger - 1 \rangle \quad (3.9b)$$

where λ is the Lagrange multiplier associated with the constraint $RR^\dagger = 1$. Since R is a rotor, then it has to be of grade zero and grade two, that is, $R \equiv \langle R \rangle + \langle R \rangle_2$. Let

$$L(R) = \sum_{i=1}^{\ell} B_i R A_i^\dagger + B_i^\dagger R A_i \quad (3.9c)$$

then the optimal rotor is the eigenrotator of $\langle L(R) \rangle_{0,2}$ with the largest eigenvalue. Note that the multiple grade projection operation $\langle \cdot \rangle_{0,2}$ is defined in (A.9d).

See [proof](#) on page 48.

Assume that the rotation R is given, then we can set $S_i = R(P_i)$ and consider the following cost function with respect to the translator T

$$J(T) = \sum_{i=1}^N d^2(T S_i T^\dagger, Q_i) + d^2(S_i, T^\dagger Q_i T) \quad (3.10)$$

where d^2 is given by (C.7). The minimum of the above cost measure, under certain conditions, has a closed form solution given by the following theorem.

Theorem 3.4. Let $Q_i \in \mathcal{G}_{4,1}^{k_i}$ and $S_i \in \mathcal{G}_{4,1}^{k_i}$ be multivectors of the same grade $k_i > 0$. After the minimization of the cost function (3.10), the optimal translation t obtained under the constraint $T = 1 + \frac{1}{2} \mathbf{e}_\infty t$ is given as

$$t = s_N^{-1} \sum_{i=1}^N \left\langle (C_1(S_i) + C_1(Q_i)) (C_3(Q_i) + C_4(Q_i) - C_3(S_i) - C_4(S_i))^\dagger \right\rangle_1 \quad (3.11a)$$

with

$$s_N = \sum_{i=1}^N \|C_1(S_i)\|^2 + \|C_1(Q_i)\|^2. \quad (3.11b)$$

and where C_i is given by (C.6).

See [proof](#) on page 50.

Another approach can be used to estimate the translation, in particular, without noise. Knowing the rotation R , we can express the solution to the translation via one of the correspondences. Let $\mathbf{P} \equiv \mathbf{P}_k$ and $\mathbf{Q} \equiv \mathbf{Q}_k$ for some integer k , also note that $\mathcal{P}_i = C_i(\mathbf{P}) = C_i(\mathbf{P}_k)$, $\mathcal{Q}_i = C_i(\mathbf{Q}) = C_i(\mathbf{Q}_k)$, then we can determine the translation exactly by the following theorem.

Theorem 3.5. *The translation vector $\mathbf{t}$ can be determined exactly from (3.7a) as*

$$\begin{aligned} \mathbf{t} &= (\mathcal{Q}_3 + \mathcal{Q}_4 - R(\mathcal{P}_3 + \mathcal{P}_4))R(\mathcal{P}_1^{-1}) \\ &= (\mathcal{Q}_3 + \mathcal{Q}_4 - R(\mathcal{P}_3 + \mathcal{P}_4))\mathcal{Q}_1^{-1} \end{aligned} \quad (3.12)$$

See [proof](#) on page 41. In summary, our proposal takes the first coefficient of each multivectors *i.e.*, $\mathbf{A}_i = C_1(\mathbf{P}_i)$ and $\mathbf{B}_i = C_1(\mathbf{Q}_i)$, to estimate the rotor $\widehat{\mathbf{R}}$, by solving the eigenvalue problem $\langle L(\mathbf{R}) \rangle_{0,2} = \lambda \mathbf{R}$. Then we take $\mathbf{S}_i = \widehat{\mathbf{R}} \mathbf{P}_i \widehat{\mathbf{R}}^\dagger$ to determine the translation $\hat{\mathbf{t}}$ as in (3.11). The general algorithm is described in Alg. 3.

Algorithm 3 Rigid Transformation Estimation

- 1) **Input Data:** $\mathbf{P}_1, \mathbf{P}_2, \dots, \mathbf{P}_m, \mathbf{Q}_1, \mathbf{Q}_2, \dots, \mathbf{Q}_m$
 - 2) Extract the coefficients $\mathcal{P}_{ij} = C_j(\mathbf{P}_i)$ and $\mathcal{Q}_{ij} = C_j(\mathbf{Q}_i)$ (eq. (3.8))
 - 3) Define the multilinear function $L(\mathbf{R}) = \sum_{i=1}^m \mathcal{Q}_{i1}^\dagger \mathbf{R} \mathcal{P}_{i1} + \mathcal{Q}_{i1} \mathbf{R} \mathcal{P}_{i1}^\dagger$ (eq. (3.9c))
 - 4) Compute the eigenrotators $\mathbf{R}_i$ of $\langle L(\mathbf{R}) \rangle_{0,2}$, with $i = 1, 2, \dots, 4$
 - 5) Chose the eigenrotator $\widehat{\mathbf{R}} = \mathbf{R}_{i^*}$ with the largest eigenvalue
 - 6) Apply the rotation $\widehat{\mathbf{R}}$ to $\mathcal{P}_{ij}$ by taking $\mathcal{S}_{ij} = \widehat{\mathbf{R}} \mathcal{P}_{ij} \widehat{\mathbf{R}}^\dagger$
 - 7) Either estimate the translation as $\hat{\mathbf{t}} = s_N^{-1} \sum_{i=1}^N \langle (\mathcal{S}_{i1} + \mathcal{Q}_{i1}) (\mathcal{Q}_{i3} + \mathcal{Q}_{i4} - \mathcal{S}_{i3} - \mathcal{S}_{i4})^\dagger \rangle_1$ (eq. (3.11)), with $s_N = \sum_{i=1}^N \|\mathcal{S}_{i1}\|^2 + \|\mathcal{Q}_{i1}\|^2$
or as
 - 8) $\hat{\mathbf{t}} = (\mathcal{Q}_{k3} + \mathcal{Q}_{k4} - \mathcal{S}_{k3} - \mathcal{S}_{k4})\mathcal{Q}_{k1}^{-1}$, for some $k = 1, 2, \dots, m$ (eq. (3.12))
-

4. Experimental evaluation

We apply the multivector extraction algorithm along with the multivector coefficients algorithm to the task of multivector cloud alignment. In this section we first describe the datasets and the metrics used for evaluation (Sec. 4.1). We then discuss the results with emphasis to the presence of noise and comparing our approach with other related methods (Sec. 4.2 and Sec. 4.3).

4.1. Datasets and evaluation

We used the Stanford 3D Scanning Repository [Turk and Levoy \(1994\)](#) that contains a variety of data models and includes theirs full and partial overlaps with the

corresponding ground truth transformations. Our experimental data mainly contains three models “Bunny”, “Armadillo” and “Dragon”. The evaluation is based on the following two metrics: (i) Relative Translation Error, $\text{RTE} = \|\hat{\mathbf{t}} - \mathbf{t}\|$, where $\hat{\mathbf{t}}$ is the estimated translation and $\mathbf{t}$ is the ground truth, and (ii) Relative Rotation Error $\text{RRE} = 2 \cos^{-1}(\widehat{\mathbf{R}} * \mathbf{R}^\dagger)$, where $\widehat{\mathbf{R}}$ and $\mathbf{R}$ are the estimated and ground truth rotations, respectively.

4.2. Experimental Setup

In our experimental setup we considered target and source point clouds described by the two set of vectors, $\mathbf{x}'_1, \mathbf{x}'_2, \dots, \mathbf{x}'_\ell \in \mathcal{A}_3$ and $\mathbf{y}'_1, \mathbf{y}'_2, \dots, \mathbf{y}'_\ell \in \mathcal{A}_3$, respectively. We assume the points are related as $\mathbf{y}'_i = R(\mathbf{x}'_{j_i}) + \mathbf{t} + \mathbf{n}_i$ where $\mathbf{n}_i$ is uncorrelated Gaussian noise with variance σ^2 . Since we are aiming to solve the alignment problem in CGA, we have to extend the points as $\mathbf{x}_i \equiv c(\mathbf{x}'_i)$ and $\mathbf{y}_i \equiv c(\mathbf{y}'_i)$ where c is the conformal mapping defined by (C.1a) (also, for a detailed description of this extension see C.1). We then proceed to use Alg. 1, setting $\mathbf{X}_k \equiv \mathbf{x}_k$ and $\mathbf{Y}_k \equiv \mathbf{y}_k$, to estimate the rotation and translation in a correspondence free manner. In particular we use Alg. 2 to extract the eigenmultivectors associated with the points $\mathbf{x}_i$ and $\mathbf{y}_i$ and then the coefficients algorithm described in Alg. 3, where the input are the eigenmultivectors associated with $\mathbf{x}_i$ and $\mathbf{y}_i$. We will term this algorithm as **CGA-EVD**, (CGA - EigenValueDecomposition).

The first methods to be compared are based on PCA algorithms [Celik and Ma \(2008\)](#); [Gonzalez \(2009\)](#); [Rehman and Lee \(2018\)](#). In PCA-based approaches the translation is estimated using the center of mass, and the rotation is estimated from the matrices

$$f(\mathbf{z}) = \sum_{k=1}^{\ell} \mathbf{z} \cdot (\mathbf{x}'_k - \bar{\mathbf{x}}')(\mathbf{x}'_k - \bar{\mathbf{x}}'), \quad g(\mathbf{z}) = \sum_{k=1}^{\ell} \mathbf{z} \cdot (\mathbf{y}'_k - \bar{\mathbf{y}}')(\mathbf{y}'_k - \bar{\mathbf{y}}'),$$

where

$$\bar{\mathbf{x}}' = \frac{1}{\ell} \sum_{k=1}^{\ell} \mathbf{x}'_k, \quad \bar{\mathbf{y}}' = \frac{1}{\ell} \sum_{k=1}^{\ell} \mathbf{y}'_k$$

are the centers of mass. Since we are using Vanilla Geometric Algebra to implement the PCA algorithm. We will term this algorithm as **VGA-EVD**.

To benchmark the performance of our algorithm we provide results with increasing levels of noise. Specifically, Figs. 2a, 2b, 2c, 2d and 2e show the Stanford Armadillo dataset corrupted with Gaussian noise with $\sigma \in \{0.001, 0.002, 0.005, 0.01, 0.02\}$, respectively. These figures helped us to find appropriate levels of noise to be applied to the datasets. We choose $\sigma \leq 0.01$, because for larger values the clouds become very distorted, losing their shape (Fig. 2d).

We extend our experimental evaluation including the following five algorithms: (i) ICP [Besl and McKay \(1992\)](#), (ii) DCP [Wang and Solomon \(2019\)](#), (iii) Go-ICP [Yang et al. \(2015\)](#), (iv) TEASER++ [Yang et al. \(2020b\)](#), and (v) PASTA [Marchi et al. \(2023\)](#). The first three algorithms are very well known in the literature, while TEASER++ [Yang et al. \(2020b\)](#) uses graduated non-convexity (GNC) [Yang et al. \(2020a\)](#) to estimate the rotation without solving a large semidefinite programming (SDP). It is proposed to deal with large amounts of outlier correspondences. PASTA [Marchi et al. \(2023\)](#), is originally proposed for estimating the rigid transformation that relate the robot’s current pose to the fixed reference frame from the LiDAR measurements. See 4.4 for the implementation details used in the experiments.

4.3. Results

Figures 3 and 4 show another important information provided by the CGA: the primitives. The primitives shown express the geometrical representation of the eigenmultivectors $\mathbf{P}_i$, $i = 1, \dots, m$, obtained by the extraction algorithm in Alg. 2. In particular, the circles shown in Figs. 3b and 4b express the eigenbivectors which are well represented by dual circles and dual imaginary circles. The spheres in Figs. 3a and 4a are the eigenvectors, which can be represented by dual spheres and dual imaginary spheres (for a more detailed description we refer to ([Dorst et al., 2007](#), Chapter 14)). Figs. 3c and 4c display arrows which represent the translation invariant components of the circles that are used to estimate the rotation between the point clouds.

Figure 5 shows all datasets with non-registered noisy input point clouds, and their respective circle primitives, *i.e.* the bivectors $\langle \mathbf{P}_i \rangle_2$ and $\langle \mathbf{Q}_i \rangle_2$, expressed as circles, computed from the eigendecomposition of F and G , respectively (see eq. (3.2)). It also illustrates the equivariant properties of the eigenmultivectors. Specifically, when the points suffer a rotation and a translation, the corresponding primitives will also suffer the same transformation. Fig. 6 illustrates the resulted alignment after applying our alignment algorithm along with the primitives of the respective point clouds. It is seen that, although difficult to visualize the alignment due to the high levels of noise, the primitives indicate a good insight of the resulted alignment. Whereby we can better visualize that the alignment was successful by looking at the primitives of both point clouds.

Figures 7, 8 and 9, show, for a level of noise $\sigma = 0.01$, the alignment results for Stanford Bunny, Armadillo, Dragon, respectively. It is shown the initial displacement of the two noisy point clouds (in (a)), with the alignment results (in (c)). For better visualization purposes, it is shown the initialization (in (b)) and the alignment results (in (d)) if the noise were removed. These figures also report

the high quantitative performance of the alignment as our framework provides low RRE and RTE scores.

Table 2 shows a comparison evaluation for the seven methods mentioned in Sec. 4.2 and for all the datasets. The results are obtained for the highest noise level, *i.e.* $\sigma = 0.01$, considering a rotation of $\theta = 5^\circ$ and translation of $\|\mathbf{t}\| = 0.01$. A detailed assessment, comprising multiple levels of noise are shown in the Tab. 3, 4 and 5 ($\theta = 5^\circ$, $\|\mathbf{t}\| = 0.01$). Tab. 6, 7 and 8, report the results in a more difficult scenario, *i.e.* $\theta \in [0^\circ, 360^\circ]$, $\|\mathbf{t}\| = 1$. From the results it is seen that the proposed **CGA-EVD** is on par with **VGA-EVD**. Notice that, the marginal superiority of the **VGA-EVD** is due to the fact that the **CGA-EVD** mapping distorts the noise, causing it to have a generalized chi-squared distribution (see *Theorem C.1*). Note that both the **CGA-EVD** and **VGA-EVD** algorithms are insensitive to the initial pose of the point clouds as their performances have negligible variation within setups.

Figures 1 and 10 compare all methods showing the alignment performance in terms of the RTE and RRE metrics, for increasing values of noise. Fig. 1 considers the setup $\theta = 5^\circ$, $\|\mathbf{t}\| = 0.01$, while Fig. 10 considers the setup $\theta \in [0^\circ, 360^\circ]$, $\|\mathbf{t}\| = 1$. It is clear that our method is robust to all values in the rotation range while local methods, *i.e.* ICP and Go-ICP, only perform well for small rotation angles and translation magnitudes. Notice, however, that even for a small transformation setup ($\theta = 5^\circ$, $\|\mathbf{t}\| = 0.01$), the performance of these local methods degrade with the increase of the noise, since the erroneous correspondences will increase as well. Concerning the **VGA-EVD**, it is shown that it achieves comparable results. The TEASER++[Yang et al. \(2020b\)](#) approach decreases its performance for higher levels of noise (see Fig. 1 for the Armadillo and Dragon datasets), as this approach gives more emphasis on the presence of outliers and not for high levels of noise. Concerning DCP, it is shown that this method provides less resilience to noise (see Figs. 1, 10), this might be due to the fact that the 3D-Match dataset [Zeng et al. \(2017\)](#) was used in the pre-training stage.

The inference time (in seconds) is reported in Tab. 9. Although our proposal is competitive, the PASTA [Marchi et al. \(2023\)](#) exhibits the overall best results.

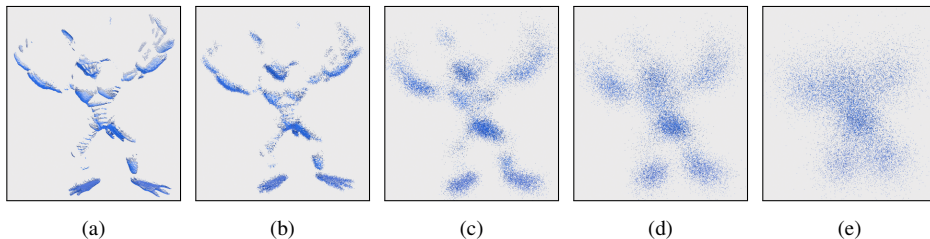


Figure 2: The Stanford Armadillo dataset for increasing levels of Gaussian Noise, *i.e.* $\sigma \in \{0, 0.001, 0.005, 0.01, 0.02\}$

Table 2: Performance analysis for all methodologies using three datasets and for the highest level of noise $\sigma = 0.01$. Benchmark results for rotation angle of $\theta = 5^\circ$ and translation magnitude $\|t\| = 0.01$. The values refer to the *mean* over 10 iterations for the same level of noise.

Algorithm	Bunny		Armadillo		Dragon	
	RRE ($^\circ$)	RTE (m)	RRE ($^\circ$)	RTE (m)	RRE ($^\circ$)	RTE (m)
VGA-EVD	7.620×10^{-1}	9.245×10^{-4}	2.889×10^{-1}	3.923×10^{-4}	1.485×10^{-1}	4.319×10^{-4}
CGA-EVD	7.381×10^{-1}	9.018×10^{-4}	2.747×10^{-1}	3.537×10^{-4}	1.462×10^{-1}	4.356×10^{-4}
ICP Besl and McKay (1992)	2.994	4.829×10^{-3}	1.697	4.779×10^{-3}	2.919	7.262×10^{-3}
PASTA-3D Marchi et al. (2023)	8.021	1.031×10^{-2}	1.228×10^1	2.580×10^{-2}	3.085	7.459×10^{-3}
DCP Wang and Solomon (2019)	2.374×10^1	2.882×10^{-2}	6.820×10^1	8.742×10^{-2}	5.306×10^1	1.009×10^{-1}
Go-ICP Yang et al. (2015)	3.553	6.434×10^{-3}	2.897	7.133×10^{-3}	3.911	9.215×10^{-3}
TEASER++ Yang et al. (2020b)	2.163×10^1	3.169×10^{-2}	8.149	1.385×10^{-2}	1.119×10^2	1.605×10^{-1}

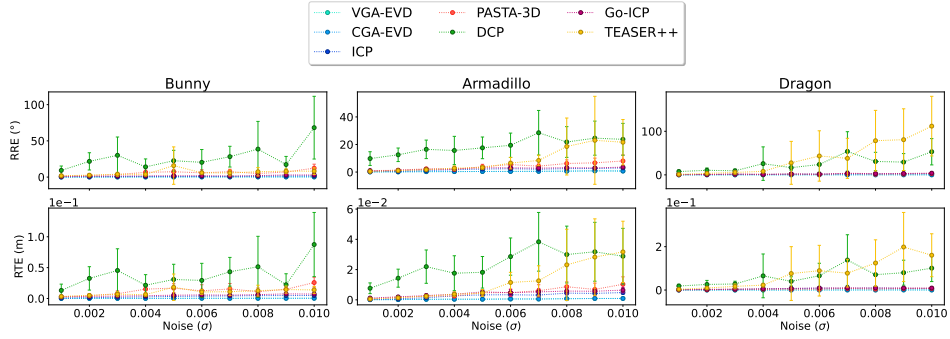


Figure 1: Benchmark results for rotation angle of $\theta = 5^\circ$ and translation magnitude $\|t\| = 0.01$ of the **VGA-EVD**, **CGA-EVD**, **ICP** Besl and McKay (1992), **PASTA-3D** Marchi et al. (2023), **DCP** Wang and Solomon (2019), **Go-ICP** Yang et al. (2015), **TEASER++** Yang et al. (2020b) methods for the Bunny, Armadillo and Dragon datasets. Error in the rotation (top row), and translation (bottom row).

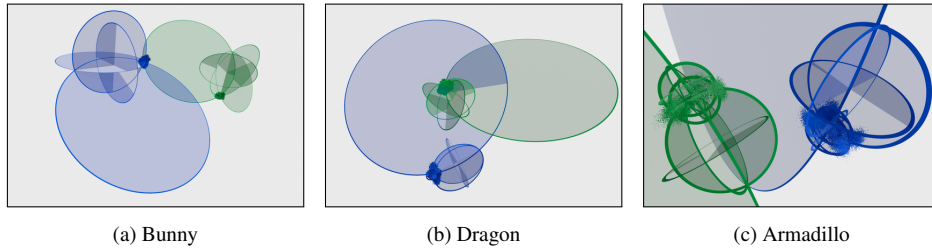


Figure 5: Multiple non-registered point clouds of the Stanford dataset with added Gaussian noise, $\sigma = 0.01$, and their respective circle primitives.

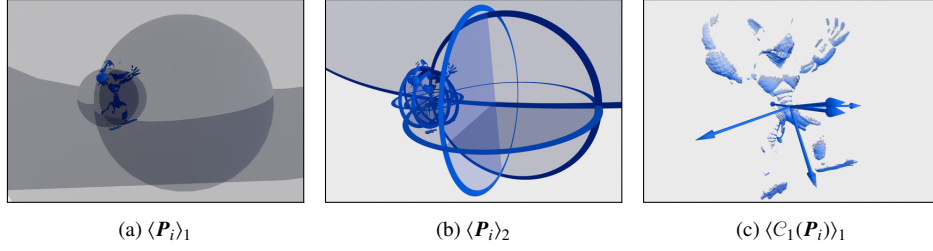


Figure 3: The Stanford Armadillo dataset and its respective primitives: (a) eigenvectors, (b) eigenbivectors, (c) translation invariant vectors extracted from the eigenbivectors.

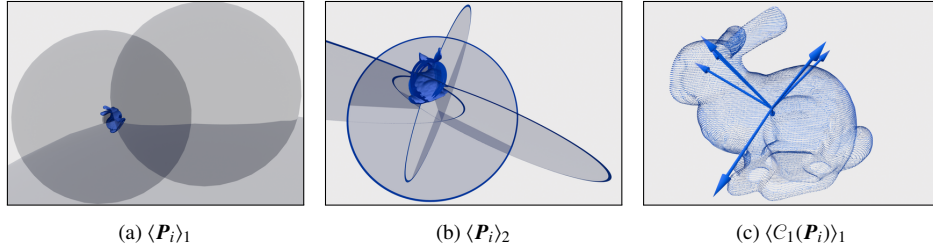


Figure 4: The Stanford Bunny dataset and its respective primitives: (a) eigenvectors, (b) eigenbivectors, (c) translation invariant vectors extracted from the eigenbivectors.

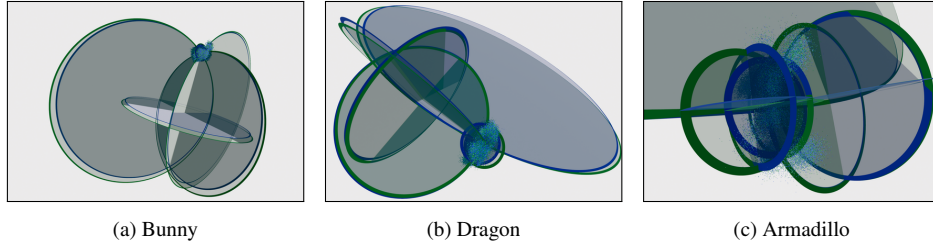


Figure 6: Multiple registered point clouds of the Stanford dataset with added Gaussian noise, $\sigma = 0.01$, and their respective circle primitives.

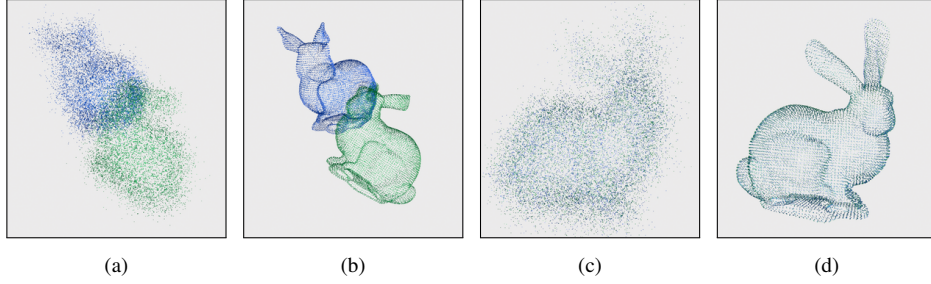


Figure 7: alignment of the Stanford Bunny. **(a)** Unregistered noisy point clouds with $\sigma = 0.01$. **(b)** Unregistered point clouds with noise removed. **(c)** Registered noisy point clouds with $\sigma = 0.01$. **(d)** Registered point clouds with noise removed. The obtained metric scores were $RRE = 9.659 \times 10^{-1}^\circ$, $RTE = 9.528 \times 10^{-1}m$.

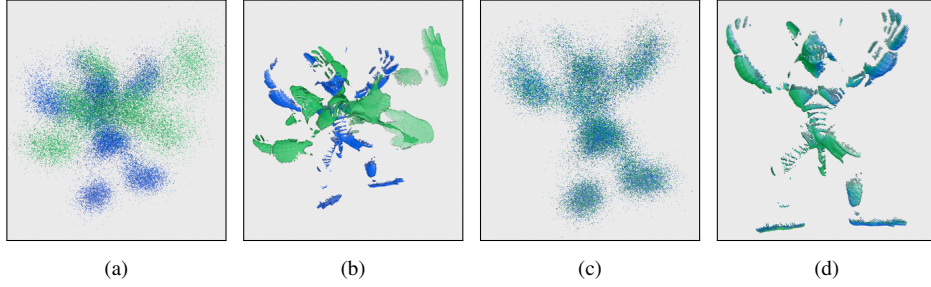


Figure 8: alignment of the Stanford Armadillo. **(a)** Unregistered noisy point clouds with $\sigma = 0.01$. **(b)** Unregistered point clouds with noise removed. **(c)** Registered noisy point clouds with $\sigma = 0.01$. **(d)** Registered point clouds with noise removed. The obtained metric scores were $RRE = 2.67 \times 10^{-1}^\circ$, $RTE = 1.932 \times 10^{-3}m$.

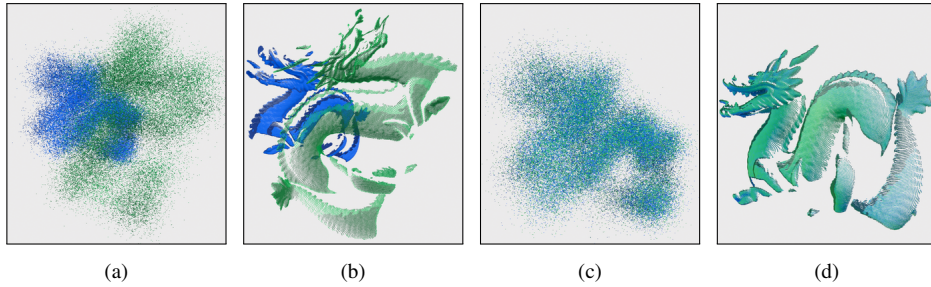


Figure 9: alignment of the Stanford Dragon. **(a)** Unregistered noisy point clouds with $\sigma = 0.01$. **(b)** Unregistered point clouds with noise removed. **(c)** Registered noisy point clouds with $\sigma = 0.01$. **(d)** Registered point clouds with noise removed. The obtained metric scores were $RRE = 3.21 \times 10^{-1}^\circ$, $RTE = 3.660 \times 10^{-3}m$.

4.4. Comparison with other related point cloud alignment methods

We provide a quantitative comparison of the methodologies and we detail some minor implementation details of the algorithms.

The comparative results are framed into the following experimental setups:

- Tables 3, 4, 5 report the results for the setup with $\theta = 5^\circ$, $\|t\| = 0.01$
- Tables 6, 7, 8 report the results for the setup with $\theta \in [0^\circ, 360^\circ]$, $\|t\| = 1$
- From the Tab. 3 to Tab. 8, it is shown that the **VGA-EVD** has a marginal superiority comparing to **CGA-EVD**. This marginal superiority is due to the fact that the **CGA-EVD** mapping distorts the noise, causing it to have a generalized chi squared distribution (see Theorem C.1). It also applies a non linear operation to the input points x'_i and y'_i , which, because of accumulated error due to floating point precision, degrades performance.

Concerning the code details of the methodologies used in Sec. 4 and also for the next experiments, they are as follows:

- For the ICP [Besl and McKay \(1992\)](#), we use the Open3D library. Concerning the hyper-parameter, we use a convergence criteria relative fitness of 1×10^{-6} and relative RMSE of 1×10^{-6} . The number of iterations is 30. These hyperparameters are used for all the datasets.
- For Go-ICP [Yang et al. \(2015\)](#), we use the python library available from the authors. The Go-ICP method is based on a branch-and-bound (BnB) strategy, whose threshold is set to $1 \times 10^{-3}N$, with N , the number of data points.
- In PASTA [Marchi et al. \(2023\)](#) although is originally proposed for 2D scenario, we use a version provided from the authors that is appropriate for the 3D setting. As in our approach there is no need of having hyperparameters. Essentially, the convex hull is estimated for each point cloud in the first stage. This is followed by the computation of the first and second order moments and the computation of the eigenvectors for the second moment.
- For TEASER++ [Yang et al. \(2020b\)](#), we have used the FPFH algorithm (provided by Open3D) to extract features and find correspondences, by which we then feed the point pair correspondences to the TEASER++ algorithm. We have also changed some of the hyperparameters, namely for the Stanford Bunny and Armadillo dataset. Concretely, we set the noise bound to 0.01 for Bunny and Armadillo datasets and for the Dragon dataset we set it to 0.02,

as these values provided the best accuracy. For the Graduated Non-Convex-Geman McClure (GNC-MG) we follow [Yang et al. \(2020a\)](#), and used a fix annealing factor of 1.4, for all the datasets.

- For DCP [Wang and Solomon \(2019\)](#), we used the DCP-v2 version with the standard architecture. In concrete, we use five Dynamic Graph CNN layers. The number of filters for each layer are respectively given by [64, 64, 128, 256, 512]. For the Transformer layer, the number of heads in multi-head attention is four with the embedding dimension of 1024. We use LayerNorm (without Dropout) with Adam optimizer and with an initial learning rate of 0.001. A total of 250 epochs were used for training. The 3D Match dataset was used for the pretraining stage. Also, because of GPU memory issues, we applied an every k points sampling before feeding the points to the algorithm. For the Bunny dataset we find that there was no need to sampling, while for the Dragon and Armadillo we set $k = 10$ and $k = 5$, respectively.

Table 3: Benchmark results for rotation angle of $\theta = 5^\circ$ and translation magnitude $\|t\| = 0.01$ for the Stanford Bunny dataset for various levels of noise. The values report the *mean* over 10 iterations for the same level of noise.

Algorithm	$\sigma = 0.001$		$\sigma = 0.002$		$\sigma = 0.005$		$\sigma = 0.01$	
	RRE ($^\circ$)	RTE (m)	RRE ($^\circ$)	RTE (m)	RRE ($^\circ$)	RTE (m)	RRE ($^\circ$)	RTE (m)
<i>VGA-EVD</i>	9.594×10^{-2}	1.076×10^{-4}	1.368×10^{-1}	1.574×10^{-4}	4.196×10^{-1}	4.299×10^{-4}	7.620×10^{-1}	9.245×10^{-4}
<i>CGA-EVD</i>	9.391×10^{-2}	1.113×10^{-4}	1.288×10^{-1}	1.600×10^{-4}	4.147×10^{-1}	4.207×10^{-4}	7.381×10^{-1}	9.018×10^{-4}
<i>ICP</i> Besl and McKay (1992)	2.727×10^{-1}	2.686×10^{-4}	5.247×10^{-1}	6.764×10^{-4}	2.170	3.297×10^{-3}	2.994	4.829×10^{-3}
<i>PASTA-3D</i> Marchi et al. (2023)	6.178×10^{-1}	6.754×10^{-4}	1.371	1.603×10^{-3}	3.830	4.685×10^{-3}	8.021	1.031×10^{-2}
<i>DCP</i> Wang and Solomon (2019)	9.770	7.657×10^{-3}	1.252×10^1	1.432×10^{-2}	1.756×10^1	1.821×10^{-2}	2.374×10^1	2.882×10^{-2}
<i>Go-ICP</i> Yang et al. (2015)	1.086	1.201×10^{-3}	1.227	1.961×10^{-3}	3.110	5.239×10^{-3}	3.553	6.434×10^{-3}
<i>TEASER++</i> Yang et al. (2020b)	5.021×10^{-1}	5.403×10^{-4}	1.045	1.345×10^{-3}	3.252	4.526×10^{-3}	2.163×10^1	3.169×10^{-2}

Table 4: Benchmark results for rotation angle of $\theta = 5^\circ$ and translation magnitude $\|t\| = 0.01$ for the Stanford Armadillo dataset for various levels of noise. The values report the *mean* over 10 iterations for the same level of noise.

Algorithm	$\sigma = 0.001$		$\sigma = 0.002$		$\sigma = 0.005$		$\sigma = 0.01$	
	RRE ($^\circ$)	RTE (m)	RRE ($^\circ$)	RTE (m)	RRE ($^\circ$)	RTE (m)	RRE ($^\circ$)	RTE (m)
<i>VGA-EVD</i>	3.153×10^{-2}	3.944×10^{-5}	5.227×10^{-2}	7.543×10^{-5}	1.410×10^{-1}	1.929×10^{-4}	2.889×10^{-1}	3.923×10^{-4}
<i>CGA-EVD</i>	3.252×10^{-2}	4.537×10^{-5}	5.785×10^{-2}	9.648×10^{-5}	1.440×10^{-1}	2.230×10^{-4}	2.747×10^{-1}	3.537×10^{-4}
<i>ICP</i> Besl and McKay (1992)	3.387×10^{-1}	8.484×10^{-4}	5.654×10^{-1}	1.462×10^{-3}	9.450×10^{-1}	3.323×10^{-3}	1.697	4.779×10^{-3}
<i>PASTA-3D</i> Marchi et al. (2023)	1.194	2.773×10^{-3}	2.231	4.557×10^{-3}	7.351	1.679×10^{-2}	1.228×10^1	2.580×10^{-2}
<i>DCP</i> Wang and Solomon (2019)	9.250	1.307×10^{-2}	2.170×10^1	3.255×10^{-2}	2.259×10^1	3.079×10^{-2}	6.820×10^1	8.742×10^{-2}
<i>Go-ICP</i> Yang et al. (2015)	9.617×10^{-1}	2.744×10^{-3}	1.262	3.299×10^{-3}	1.727	5.299×10^{-3}	2.897	7.133×10^{-3}
<i>TEASER++</i> Yang et al. (2020b)	1.857	3.242×10^{-3}	2.654	4.970×10^{-3}	1.584×10^1	1.821×10^{-2}	8.149	1.385×10^{-2}

5. Conclusion

We proposed a novel correspondence free multivector cloud alignment in CGA. The distinctiveness of our proposal is characterized by the equivariant relationship

Table 5: Benchmark results for rotation angle of $\theta = 5^\circ$ and translation magnitude $\|t\| = 0.01$ for the Stanford Dragon dataset for various levels of noise. The values report the *mean* over 10 iterations for the same level of noise.

Algorithm	$\sigma = 0.001$		$\sigma = 0.002$		$\sigma = 0.005$		$\sigma = 0.01$	
	RRE ($^\circ$)	RTE (m)	RRE ($^\circ$)	RTE (m)	RRE ($^\circ$)	RTE (m)	RRE ($^\circ$)	RTE (m)
<i>VGA-EVD</i>	2.830×10^{-2}	7.308×10^{-5}	4.731×10^{-2}	1.267×10^{-4}	9.216×10^{-2}	2.268×10^{-4}	1.485×10^{-1}	4.319×10^{-4}
<i>CGA-EVD</i>	2.939×10^{-2}	7.679×10^{-5}	4.895×10^{-2}	1.308×10^{-4}	9.409×10^{-2}	2.306×10^{-4}	1.462×10^{-1}	4.356×10^{-4}
<i>ICP Besl and McKay (1992)</i>	3.629×10^{-1}	8.018×10^{-4}	6.194×10^{-1}	2.041×10^{-3}	1.715	6.741×10^{-3}	2.919	7.262×10^{-3}
<i>PASTA-3D Marchi et al. (2023)</i>	3.708×10^{-1}	1.010×10^{-3}	7.604×10^{-1}	1.749×10^{-3}	1.785	3.967×10^{-3}	3.085	7.459×10^{-3}
<i>DCP Wang and Solomon (2019)</i>	7.601	1.857×10^{-2}	1.050×10^1	2.585×10^{-2}	1.654×10^1	4.021×10^{-2}	5.306×10^1	1.009×10^{-1}
<i>Go-ICP Yang et al. (2015)</i>	8.019×10^{-1}	1.842×10^{-3}	9.176×10^{-1}	3.354×10^{-3}	2.038	7.422×10^{-3}	3.911	9.215×10^{-3}
<i>TEASER++ Yang et al. (2020b)</i>	1.321	3.640×10^{-3}	3.429	9.375×10^{-3}	2.761×10^1	7.604×10^{-2}	1.119×10^2	1.605×10^{-1}

Table 6: Benchmark results for random rotation angle and translation magnitude $\|t\| = 1$ for the Stanford Bunny dataset for various levels of noise. The values report the *mean* over 10 iterations for the same level of noise.

Algorithm	$\sigma = 0.001$		$\sigma = 0.002$		$\sigma = 0.005$		$\sigma = 0.01$	
	RRE ($^\circ$)	RTE (m)	RRE ($^\circ$)	RTE (m)	RRE ($^\circ$)	RTE (m)	RRE ($^\circ$)	RTE (m)
<i>VGA-EVD</i>	9.962×10^{-2}	1.078×10^{-4}	1.943×10^{-1}	1.668×10^{-4}	3.250×10^{-1}	4.039×10^{-4}	1.027	1.307×10^{-3}
<i>CGA-EVD</i>	1.023×10^{-1}	1.119×10^{-4}	1.897×10^{-1}	1.642×10^{-4}	3.242×10^{-1}	4.022×10^{-4}	1.009	1.360×10^{-3}
<i>ICP Besl and McKay (1992)</i>	6.580×10^1	6.044×10^{-2}	1.120×10^2	1.101×10^{-1}	1.232×10^2	1.180×10^{-1}	1.176×10^2	1.372×10^{-1}
<i>PASTA-3D Marchi et al. (2023)</i>	7.993×10^{-1}	8.760×10^{-4}	1.452	2.141×10^{-3}	3.048	3.846×10^{-3}	9.020	7.361×10^{-3}
<i>DCP Wang and Solomon (2019)</i>	7.747×10^1	4.311×10^{-2}	6.282×10^1	3.198×10^{-2}	8.137×10^1	5.097×10^{-2}	3.400×10^1	5.041×10^{-2}
<i>Go-ICP Yang et al. (2015)</i>	8.885×10^1	8.820×10^{-2}	6.234×10^1	6.903×10^{-2}	1.265×10^2	1.197×10^{-1}	6.797×10^1	8.158×10^{-2}
<i>TEASER++ Yang et al. (2020b)</i>	4.813	2.185×10^{-3}	2.065	2.387×10^{-3}	2.420×10^1	3.112×10^{-2}	3.567×10^1	3.894×10^{-2}

between the eigenmultivectors between the source and target multivector clouds. This enables a robust identification of the transformation between the eigenmultivectors which in turn extends to the multivector clouds. We have shown how our approach is robust to high levels of noise and how it compares with similar approaches, such as the PASTA and PCA based algorithms, (*i.e.* *VGA-EVD*). Further work will address the problem of low overlap between the two point clouds. This will be achieved by extending the notion of eigenmultivectors to some specific region in the multivector cloud. Using this approach and by dividing the point cloud into regions could provide a robust rotation and translation equivariant de-

Table 7: Benchmark results for random rotation angle and translation magnitude $\|t\| = 1$ for the Stanford Armadillo dataset for various levels of noise. The values report the *mean* over 10 iterations for the same level of noise.

Algorithm	$\sigma = 0.001$		$\sigma = 0.002$		$\sigma = 0.005$		$\sigma = 0.01$	
	RRE ($^\circ$)	RTE (m)	RRE ($^\circ$)	RTE (m)	RRE ($^\circ$)	RTE (m)	RRE ($^\circ$)	RTE (m)
<i>VGA-EVD</i>	1.755×10^{-2}	2.598×10^{-5}	6.899×10^{-2}	9.089×10^{-5}	9.812×10^{-2}	1.553×10^{-4}	2.638×10^{-1}	3.623×10^{-4}
<i>CGA-EVD</i>	1.722×10^{-2}	2.916×10^{-5}	7.280×10^{-2}	1.005×10^{-4}	1.056×10^{-1}	1.785×10^{-4}	3.037×10^{-1}	3.689×10^{-4}
<i>ICP Besl and McKay (1992)</i>	8.861×10^1	1.438×10^{-1}	9.736×10^1	1.635×10^{-1}	8.024×10^1	1.218×10^{-1}	1.021×10^2	1.426×10^{-1}
<i>PASTA-3D Marchi et al. (2023)</i>	1.318	3.025×10^{-3}	1.713	4.057×10^{-3}	6.381	1.436×10^{-2}	9.846	2.059×10^{-2}
<i>DCP Wang and Solomon (2019)</i>	1.937×10^1	2.830×10^{-2}	5.132×10^1	6.084×10^{-2}	3.634×10^1	5.294×10^{-2}	3.945×10^1	5.552×10^{-2}
<i>Go-ICP Yang et al. (2015)</i>	9.449×10^1	1.778×10^{-1}	7.929×10^1	1.245×10^{-1}	6.117×10^1	1.149×10^{-1}	7.724×10^1	9.612×10^{-2}
<i>TEASER++ Yang et al. (2020b)</i>	2.295	3.851×10^{-3}	1.430×10^1	1.824×10^{-2}	2.127×10^1	2.299×10^{-2}	2.232×10^1	2.790×10^{-2}

Table 8: Benchmark results for random rotation angle and translation magnitude $\|t\| = 1$ for the Stanford Dragon dataset for various levels of noise. The values report the *mean* over 10 iterations for the same level of noise.

Algorithm	$\sigma = 0.001$		$\sigma = 0.002$		$\sigma = 0.005$		$\sigma = 0.01$	
	RRE ($^\circ$)	RTE (m)	RRE ($^\circ$)	RTE (m)	RRE ($^\circ$)	RTE (m)	RRE ($^\circ$)	RTE (m)
VGA-EVD	1.933×10^{-2}	4.375×10^{-5}	4.442×10^{-2}	1.063×10^{-4}	1.082×10^{-1}	3.268×10^{-4}	1.729×10^{-1}	3.529×10^{-4}
CGA-EVD	1.916×10^{-2}	4.617×10^{-5}	4.401×10^{-2}	1.096×10^{-4}	1.143×10^{-1}	3.474×10^{-4}	1.689×10^{-1}	3.706×10^{-4}
ICP Besl and McKay (1992)	3.769×10^1	6.513×10^{-2}	6.589×10^1	8.795×10^{-2}	6.080×10^1	7.849×10^{-2}	6.074×10^1	1.244×10^{-1}
PASTA-3D Marchi et al. (2023)	4.737×10^{-1}	1.147×10^{-3}	7.842×10^{-1}	2.088×10^{-3}	2.057	3.693×10^{-3}	4.502	4.413×10^{-3}
DCP Wang and Solomon (2019)	7.939	2.230×10^{-2}	1.211×10^1	3.634×10^{-2}	3.319×10^1	8.802×10^{-2}	5.787×10^1	1.352×10^{-1}
Go-ICP Yang et al. (2015)	3.764×10^1	5.438×10^{-2}	4.609×10^1	6.590×10^{-2}	8.674×10^1	1.139×10^{-1}	5.424×10^1	9.500×10^{-2}
TEASER++ Yang et al. (2020b)	3.029	7.468×10^{-3}	2.832×10^1	3.789×10^{-2}	7.092×10^1	1.456×10^{-1}	8.233×10^1	1.760×10^{-1}

Table 9: Mean inference time (in seconds) for the Stanford Bunny, Armadillo and Dragon datasets for the different algorithms.

Algorithm	Bunny	Armadillo	Dragon
VGA-EVD	9.280×10^{-2} s	2.014×10^{-1} s	3.805×10^{-1} s
CGA-EVD	1.619×10^{-1} s	3.749×10^{-1} s	6.831×10^{-1} s
ICP Besl and McKay (1992)	5.491×10^{-2} s	1.008×10^{-1} s	1.816×10^{-1} s
PASTA-3D Marchi et al. (2023)	2.453×10^{-2} s	5.628×10^{-2} s	1.022×10^{-1} s
DCP Wang and Solomon (2019)	8.816 s	1.751 s	1.493 s
Go-ICP Yang et al. (2015)	5.794 s	5.797 s	6.040 s
TEASER++ Yang et al. (2020b)	1.617 s	6.148 s	9.264 s

scriptors (the eigenmultivectors), as well as invariant descriptors (the eigenvalues) for multivector cloud data.

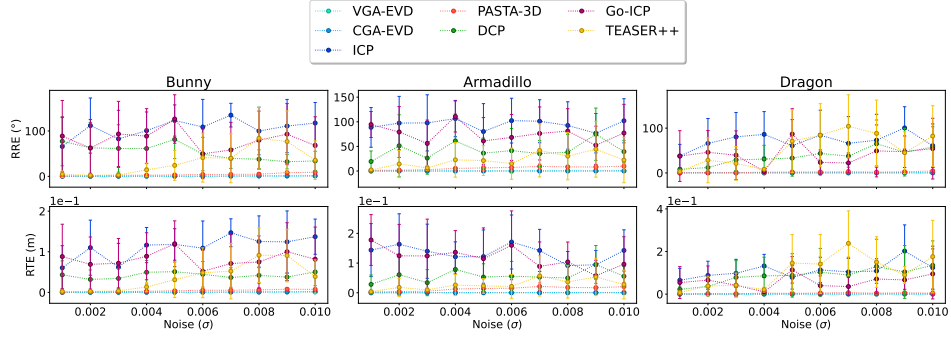


Figure 10: Benchmark results for random rotation angle and translation magnitude $\|t\| = 1$ of the *VGA-EVD*, *CGA-EVD*, *ICP* Besl and McKay (1992), *PASTA-3D* Marchi et al. (2023), *DCP* Wang and Solomon (2019), *Go-ICP* Yang et al. (2015), *TEASER++* Yang et al. (2020b) methods for the Bunny, Armadillo and Dragon datasets. Error in the rotation (top row), and translation (bottom row).

Supplementary material

Additional proofs and algorithmic details are provided in the Appendix. These materials are available in the supplementary files submitted with the manuscript and at the following repository: <https://github.com/anon-69420/Appendix>

Code and Data availability

The code is available at:

<https://github.com/anon-69420/RegistrationGA>

The Stanford 3D Scanning Repository Turk and Levoy (1994) is available at the URL <http://www.graphics.stanford.edu/data/3Dscanrep/>.

Declaration of competing interests

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Declaration of generative AI and AI-assisted technologies in the manuscript preparation process

The authors declare that no generative AI or AI-assisted technologies were used in the preparation of this manuscript.

References

- Agarwal, S., Furukawa, Y., Snavely, N., Simon, I., Curless, B., Seitz, S.M., Szeliski, R., 2011. Building rome in a day. *Communications of the ACM* 54, 105–112.
- Aoki, Y., Goforth, H., Srivatsan, R.A., Lucey, S., 2019. Pointnetlk: Robust & efficient point cloud registration using pointnet, in: *CVPR*.
- Artin, E., 2016. *Geometric algebra*. Courier Dover Publications.
- Bai, X., Luo, Z., Zhou, L., Fu, H., Quan, L., Tai, C.L., 2020. D3feat: Joint learning of dense detection and description of 3d local features, in: *CVPR*.
- Barath, D., Matas, J., 2018. Graph-cut ransac, in: *ICCV*.
- Besl, P.J., McKay, N.D., 1992. Method for registration of 3-d shapes, in: *Sensor fusion IV: control paradigms and data structures*, Spie. pp. 586–606.
- Bustos, A.P., Chin, T.J., 2017. Guaranteed outlier removal for point cloud registration with correspondences. *IEEE PAMI* 40, 2868–2882.
- Celik, T., Ma, K.K., 2008. Fast object-based image registration using principal component analysis for super-resolution imaging, in: *2008 5th International Conference on Visual Information Engineering (VIE 2008)*.
- Chen, Z., Sun, K., Yang, F., Tao, W., 2022. Sc2-pcr: A second order spatial compatibility for efficient and robust point cloud registration, in: *CVPR*.
- Choy, C., Dong, W., Koltun, V., 2020. Deep global registration, in: *CVPR*.
- Choy, C., Park, J., Koltun, V., 2019. Fully convolutional geometric features, in: *ICCV*.
- Clifford, 1871. Preliminary sketch of biquaternions. *Proceedings of the London Mathematical Society* 1, 381–395.
- Dang, Z., Wang, L., Guo, Y., Salzmann, M., 2022. Learning-based point cloud registration for 6d object pose estimation in the real world, in: *ECCV*.
- Deng, H., Birdal, T., Ilic, S., 2018. Ppf-foldnet: Unsupervised learning of rotation invariant 3d local descriptors, in: *ECCV*.
- Deschaud, J.E., 2018. Imls-slam: Scan-to-model matching based on 3d data, in: *ICRA, IEEE*.

- Dorst, L., Fontijne, D., Mann, S., 2007. Geometric algebra for computer science: an object-oriented approach to geometry. Elsevier ; Morgan Kaufmann, Amsterdam, San Francisco. URL: <http://geometricalgebra.net/>. oCLC: 182548505.
- Dorst, L., Mann, S., 2002. Geometric algebra: a computational framework for geometrical applications. IEEE Computer Graphics and Applications 22, 24–31.
- Fischler, M.A., Bolles, R.C., 1981. Random sample consensus: a paradigm for model fitting with applications to image analysis and automated cartography. Communications of the ACM 24, 381–395.
- Frome, A., Huber, D., Kolluri, R., Bülow, T., Malik, J., 2004. Recognizing objects in range data using regional point descriptors, in: ECCV, Springer.
- Fu, K., Liu, S., Luo, X., Wang, M., 2021. Robust point cloud registration framework based on deep graph matching, in: CVPR.
- Gonzalez, R.C., 2009. Digital image processing. Pearson education india.
- Hestenes, D., 2015. Space-time algebra. Springer.
- Hestenes, D., Sobczyk, G., 1984. Clifford Algebra to Geometric Calculus. Springer Netherlands, Dordrecht. URL: <http://link.springer.com/10.1007/978-94-009-6292-7>, doi:10.1007/978-94-009-6292-7.
- Huang, S., Gojcic, Z., Usvyatsov, M., Wieser, A., Schindler, K., 2021. Predator: Registration of 3d point clouds with low overlap, in: CVPR.
- Huang, X., Mei, G., Zhang, J., 2020. Feature-metric registration: A fast semi-supervised approach for robust point cloud registration without correspondences, in: CVPR.
- Le, H.M., Do, T.T., Hoang, T., Cheung, N.M., 2019. Sdrsac: Semidefinite-based randomized approach for robust point cloud registration without correspondences, in: ICCV.
- Leordeanu, M., Hebert, M., 2005. A spectral technique for correspondence problems using pairwise constraints, in: ICCV, IEEE.
- Li, J., Hu, Q., Ai, M., 2020a. Gesac: Robust graph enhanced sample consensus for point cloud registration. ISPRS Journal of Photogrammetry and Remote Sensing 167, 363–374.

- Li, J., Zhang, C., Xu, Z., Zhou, H., Zhang, C., 2020b. Iterative distance-aware similarity matrix convolution with mutual-supervised point elimination for efficient point cloud registration, in: ECCV, Springer.
- Li, X., Pontes, J.K., Lucey, S., 2021. Pointnetlk revisited, in: CVPR.
- Li, X., Wang, L., Fang, Y., 2019. Pc-net: Unsupervised point correspondence learning with neural networks, in: 3DV, IEEE.
- Li, X., Wang, L., Fang, Y., 2020c. Unsupervised partial point set registration via joint shape completion and registration. arXiv preprint arXiv:2009.05290 .
- Li, Y., Harada, T., 2022. Leopard: Learning partial point cloud matching in rigid and deformable scenes. ieee, in: CVPR.
- Lowe, D.G., 2004. Distinctive image features from scale-invariant keypoints. IJCV 60.
- Lu, F., Chen, G., Liu, Y., Zhang, L., Qu, S., Liu, S., Gu, R., 2021. Hregnet: A hierarchical network for large-scale outdoor lidar point cloud registration, in: ICCV.
- Lu, W., Wan, G., Zhou, Y., Fu, X., Yuan, P., Song, S., 2019. Deepvcv: An end-to-end deep neural network for point cloud registration, in: ICCV.
- Lucas, B.D., Kanade, T., 1981. An iterative image registration technique with an application to stereo vision, in: IJCAI, pp. 674–679.
- Marchi, M., Bunton, J., Gas, Y., Gharesifard, B., Tabuada, P., 2023. Sharp performance bounds for pasta. IEEE Control Systems Letters .
- Pepe, A., Lasenby, J., 2023. Cga-posenet: Camera pose regression via a 1d-up approach to conformal geometric algebra. arXiv preprint arXiv:2302.05211 .
- Pepe, A., Lasenby, J., Buchholz, S., 2024. Cgaposenet+ gcan: A geometric clifford algebra network for geometry-aware camera pose regression, in: WACV, pp. 6593–6603.
- Pillai, S.S., Megalingam, R.K., 2020. Detection and 3d modeling of brain tumor using machine learning and conformal geometric algebra, in: 2020 Int. Conf. on Communication and Signal Processing (ICCSP), IEEE.
- Qi, C.R., Su, H., Mo, K., Guibas, L.J., 2017. Pointnet: Deep learning on point sets for 3d classification and segmentation, in: CVPR.

- Qin, Z., Yu, H., Wang, C., Guo, Y., Peng, Y., Xu, K., 2022. Geometric transformer for fast and robust point cloud registration, in: CVPR.
- Rehman, H.Z.U., Lee, S., 2018. Automatic image alignment using principal component analysis. *IEEE Access* 6, 72063–72072.
- Ruhe, D., Brandstetter, J., Forré, P., 2023a. Clifford group equivariant neural networks. *arXiv preprint arXiv:2305.11141* .
- Ruhe, D., Gupta, J.K., De Keninck, S., Welling, M., Brandstetter, J., 2023b. Geometric clifford algebra networks. *arXiv preprint arXiv:2302.06594* .
- Rusu, R.B., Blodow, N., Beetz, M., 2009. Fast point feature histograms (fpfh) for 3d registration, in: ICRA, IEEE.
- Salti, S., Tombari, F., Di Stefano, L., 2014. Shot: Unique signatures of histograms for surface and texture description. *CVIU* 125.
- Schonberger, J.L., Frahm, J.M., 2016. Structure-from-motion revisited, in: CVPR.
- Turk, G., Levoy, M., 1994. Stanford 3d scanning repository. Available at <http://graphics.stanford.edu/data/3Dscanrep/>. Accessed 6 Apr 2023.
- Wang, H., Liu, Y., Dong, Z., Wang, W., 2022. You only hypothesize once: Point cloud registration with rotation-equivariant descriptors, in: Proceedings of the 30th ACM International Conference on Multimedia.
- Wang, Y., Solomon, J.M., 2019. Deep closest point: Learning representations for point cloud registration, in: ICCV, pp. 3523–3532.
- Wong, J.M., Kee, V., Le, T., Wagner, S., Mariottini, G.L., Schneider, A., Hamilton, L., Chipalkatty, R., Hebert, M., Johnson, D.M., et al., 2017. Segicp: Integrated deep semantic segmentation and pose estimation, in: IROS, IEEE.
- Yang, H., Antonante, P., Tzoumas, V., Carlone, L., 2020a. Graduated non-convexity for robust spatial perception: From non-minimal solvers to global outlier rejection. *IEEE Robotics and Automation Letters* 5, 1127–1134.
- Yang, H., Shi, J., Carlone, L., 2020b. Teaser: Fast and certifiable point cloud registration. *IEEE Transactions on Robotics* 37, 314–333.
- Yang, J., Li, H., Campbell, D., Jia, Y., 2015. Go-icp: A globally optimal solution to 3d icp point-set registration. *IEEE transactions on pattern analysis and machine intelligence* 38, 2241–2254.

- Yang, J., Xian, K., Wang, P., Zhang, Y., 2019. A performance evaluation of correspondence grouping methods for 3d rigid data matching. *IEEE PAMI* 43, 1859–1874.
- Yew, Z.J., Lee, G.H., 2020. Rpm-net: Robust point matching using learned features, in: *CVPR*.
- Yuan, W., Eckart, B., Kim, K., Jampani, V., Fox, D., Kautz, J., 2020. Deepgmr: Learning latent gaussian mixture models for registration, in: *ECCV*, Springer.
- Zeng, A., Song, S., Nießner, M., Fisher, M., Xiao, J., Funkhouser, T., 2017. 3dmatch: Learning local geometric descriptors from rgb-d reconstructions, in: *Proceedings of the IEEE conference on computer vision and pattern recognition*, pp. 1802–1811.
- Zhang, J., Singh, S., 2014. Loam: Lidar odometry and mapping in real-time., in: *Robotics: Science and systems*, pp. 1–9.
- Zhou, Q.Y., Park, J., Koltun, V., 2016. Fast global registration, in: *ECCV*, Springer.
- Zhu, J., Fang, Y., 2020. Reference grid-assisted network for 3d point signature learning from point clouds, in: *CVPR*.

Introduction to the Appendix

This appendix is structured as follows:

- Section [A](#) introduces Geometric Algebra (GA), with core concepts, specifying the notation which is used throughout the paper, (Sec. [3.3.2](#) of the paper).
- Section [B](#) details how the multivectors differentiation is computed and introduces differentiation as a limit definition. Also, it describes some derivatives that are further used in Section [E](#) (optimization), (Sec. [3.3.2](#) of the paper).
- Section [C](#) presents Conformal Geometric Algebra (CGA) by introducing the conformal mapping. It describes 3D translations and rotation in CGA and how to decompose a multivector in CGA into its constituent parts (the coefficients), concretizing them analytically, (Secs. [3.3](#), [3.3.1](#) [3.3.2](#), [4.2](#), [4.3](#) of the paper).
- Section [D](#) defines multilinear and differential multilinear transformations that are used to define the symmetry of these family of functions. It describes the eigendecomposition of symmetric multilinear functions, (Sec. [3.2](#) of the paper).

- Section [E](#) introduces the Lagrangian with respect to multivector constraints, *i.e.* the multivector Lagrange Multiplier. Solutions for the optimal rotor and the optimal translation problems are provided, (Sec. [3.3.2](#) of the paper).

A. Geometric Algebra

The contents described in the section are mainly based on the book by David Hestenes [Hestenes and Sobczyk \(1984\)](#). Our goal is to provide some background and theoretical cornerstones in GA used in this paper, contributing for the self-containment of the document. Also note that, this section provides some definitions that will be used in Sec. C.

Let $\mathcal{A}_{p,q}$ be a $(p + q)$ -dimensional vector space over the field of real numbers $\mathbb{R}$, with canonical vector basis

$$\underbrace{\{e_1, e_2, \dots, e_p\}}_{e_i^2=+1} \underbrace{\{e_{p+1}, e_{p+2}, \dots, e_{p+q}\}}_{e_j^2=-1} \quad (\text{A.1})$$

Also, let the product be defined in $\mathcal{A}_{p,q}$ according to

$$e_i e_j + e_j e_i = 2\eta_{ij} \quad (\text{A.2})$$

where

$$\eta_{ij} = \begin{cases} +1 & \text{if } i = j, 1 \leq i \leq p \\ -1 & \text{if } i = j, p+1 \leq i \leq p+q \\ 0 & \text{if } i \neq j \end{cases} \quad (\text{A.3})$$

The non-commutative product in (A.2) is called the geometric product, and generates the associative 2^{p+q} dimensional geometric algebra $\mathcal{G}_{p,q} = \mathcal{G}(\mathcal{A}_{p,q})$ over the vector space $\mathcal{A}_{p,q}$.

The geometric algebra $\mathcal{G}_{p,q}$ is a 2^{p+q} dimensional real linear vector space with canonical basis

$$\{e_J : J \subseteq \{1, 2, \dots, p+q\}\} \quad (\text{A.4})$$

where

$$e_J \equiv e_{j_1 j_2 \dots j_k} \equiv e_{j_1} e_{j_2} \dots e_{j_k}, \quad \text{for } 1 \leq j_1 < j_2 < \dots < j_k \leq p+q \quad (\text{A.5})$$

and $e_\emptyset = 1$. Elements of the basis $\{e_J\}$ are called basis blades.

A basis blade e_J which can be written as the geometric product of k basis vectors is said to be of grade k . This means that the geometric algebra $\mathcal{G}_{p,q}$ can be decomposed into linear subspaces of different grade, that is,

$$\mathcal{G}_{p,q} = \bigoplus_{k=0}^{p+q} \mathcal{G}_{p,q}^k. \quad (\text{A.6})$$

where the k -grade subspace is denoted by $\mathcal{G}_{p,q}^k$. Note that the scalars $\mathbb{R} \equiv \mathcal{G}_{p,q}^0$ and the vectors $\mathcal{A}_{p,q} \equiv \mathcal{G}_{p,q}^1$ are both subspaces of the geometric algebra $\mathcal{G}_{p,q}$. Geometric Algebra can also be decomposed into even grade linear subspaces $\mathcal{G}_{p,q}^+$ and odd grade linear subspaces $\mathcal{G}_{p,q}^-$, *i.e.*

$$\mathcal{G}_{p,q} = \mathcal{G}_{p,q}^+ \oplus \mathcal{G}_{p,q}^- \quad (\text{A.7})$$

where the even graded subspace $\mathcal{G}_{p,q}^+$ is on itself a subalgebra of $\mathcal{G}_{p,q}$. We can call it the *even subalgebra of $\mathcal{G}_{p,q}$* . A general element of the geometric algebra $\mathcal{G}_{p,q}$ is called a multivector. We assume that any multivector X can be written as the sum

$$X \equiv \sum_{k=0}^{p+q} \langle X \rangle_k \quad (\text{A.8})$$

where $\langle X \rangle_k$ is the grade projection operator defined as

$$\langle \cdot \rangle_k : \mathcal{G}_{p,q} \mapsto \mathcal{G}_{p,q}^k : X \mapsto \langle X \rangle_k \quad (\text{A.9a})$$

We drop the subscript when $k = 0$, *i.e.* when we do the projection to the scalar part $\langle X \rangle_0 \equiv \langle X \rangle \in \mathbb{R}$. The grade projection operator is distributive

$$\langle X + Y \rangle_k = \langle X \rangle_k + \langle Y \rangle_k, \quad \forall X, Y \in \mathcal{G}_{p,q} \quad (\text{A.9b})$$

It is associative and commutative in terms of scalar multiplication

$$\alpha \langle X \rangle_k = \langle \alpha X \rangle_k = \langle X \rangle_k \alpha, \quad \text{if } \alpha = \langle \alpha \rangle \text{ and } \forall X \in \mathcal{G}_{p,q} \quad (\text{A.9c})$$

A multivector A is said to be a k -vector if it is an element of the k -grade subspace $\mathcal{G}_{p,q}^k$. Then if $A \equiv \langle A \rangle_k$ we can call A a multivector of grade k or a k -vector. We will often write bivector to refer to 2-vectors, trivector for 3-vectors and vector to refer to 1-vectors. We will often employ the notation $\langle \cdot \rangle_{k_1, k_2, \dots, k_s}$ to refer to grade projection to multiple grades simultaneously, that is

$$\langle X \rangle_{k_1, k_2, \dots, k_s} = \langle X \rangle_{k_1} + \langle X \rangle_{k_2} + \dots + \langle X \rangle_{k_s} \quad (\text{A.9d})$$

The reversion operation † reorders the factor in a product. It can be defined by changing the order of the basis vectors of a given multivector. For a basis blade e_J it is defined as

$$(e_{j_1} e_{j_2} \dots e_{j_k})^\dagger \equiv e_{j_k} e_{j_{k-1}} \dots e_{j_1} \quad (\text{A.10})$$

From the definition above, for $\mathbf{A}, \mathbf{B} \in \mathcal{G}_{p,q}$ and $\mathbf{a} \in \mathcal{A}_{p,q}$, we can find how the reversion operation enjoys the properties

$$(\mathbf{AB})^\dagger = \mathbf{B}^\dagger \mathbf{A}^\dagger \quad (\text{A.11a})$$

$$(\mathbf{A} + \mathbf{B})^\dagger = \mathbf{A}^\dagger + \mathbf{B}^\dagger \quad (\text{A.11b})$$

$$\langle \mathbf{A}^\dagger \rangle = \langle \mathbf{A} \rangle \quad (\text{A.11c})$$

$$\mathbf{a}^\dagger = \mathbf{a} \quad \text{where } \mathbf{a} = \langle \mathbf{a} \rangle_1 \quad (\text{A.11d})$$

it follows that the reverse of a product of vectors is

$$(\mathbf{a}_1 \mathbf{a}_2 \cdots \mathbf{a}_k)^\dagger = \mathbf{a}_k \mathbf{a}_{k-1} \cdots \mathbf{a}_1 \quad (\text{A.12})$$

where $\mathbf{a}_i \in \mathcal{A}_{p,q}$.

Let $\mathbf{A}_k = \mathbf{a}_1 \mathbf{a}_2 \cdots \mathbf{a}_k$ and assume that the vectors $\mathbf{a}_i$ anti-commute, that is $\mathbf{a}_i \mathbf{a}_j = -\mathbf{a}_j \mathbf{a}_i$ for $i \neq j$, then reordering the vectors on the right hand side of (A.12) we easily prove that

$$\mathbf{A}_k^\dagger \equiv (-1)^{k(k-1)/2} \mathbf{A}_k \quad (\text{A.13})$$

Then, the reversion operator acts on a multivector $\mathbf{X}$ as

$$\mathbf{X}^\dagger = \sum_{i=0}^{p+q} (-1)^{i(i-1)/2} \langle \mathbf{X} \rangle_i \quad (\text{A.14})$$

We define the *inner product* of multivector of unique grade by

$$\mathbf{A}_r \cdot \mathbf{B}_s \equiv \langle \mathbf{A}_r \mathbf{B}_s \rangle_{|r-s|}, \quad \text{for } r, s > 0 \quad (\text{A.15a})$$

$$\mathbf{A}_r \cdot \mathbf{B}_s \equiv 0, \quad \text{if } r = 0 \text{ or } s = 0 \quad (\text{A.15b})$$

The inner product for arbitrary multivectors is then defined by

$$\mathbf{A} \cdot \mathbf{B} \equiv \sum_{i=0}^n \langle \mathbf{A} \rangle_i \cdot \mathbf{B} \equiv \sum_{j=0}^n \langle \mathbf{B} \rangle_j \cdot \mathbf{A} \equiv \sum_{i=0}^n \sum_{j=0}^n \langle \mathbf{A} \rangle_i \cdot \langle \mathbf{B} \rangle_j \quad (\text{A.15c})$$

with $n = p + q$. In a similar way, we define the *outer product* of multivectors of unique grade by

$$\mathbf{A}_r \wedge \mathbf{B}_s \equiv \langle \mathbf{A}_r \mathbf{B}_s \rangle_{r+s} \quad (\text{A.16a})$$

The *outer product* for arbitrary multivectors is then defined by

$$\mathbf{A} \wedge \mathbf{B} \equiv \sum_{i=0}^n \langle \mathbf{A} \rangle_i \wedge \mathbf{B} \equiv \sum_{j=0}^n \langle \mathbf{B} \rangle_j \wedge \mathbf{A} \equiv \sum_{i=0}^n \sum_{j=0}^n \langle \mathbf{A} \rangle_i \wedge \langle \mathbf{B} \rangle_j \quad (\text{A.16b})$$

The **scalar product** is defined as the scalar part of the geometric product of two arbitrary multivectors A, B and is defined as

$$A * B = \langle AB \rangle \quad (\text{A.17a})$$

We can show that the scalar product has the following properties

$$A^\dagger * B^\dagger = A * B \quad \Leftrightarrow \quad \langle A^\dagger B^\dagger \rangle = \langle AB \rangle \quad (\text{A.17b})$$

$$A * B = B * A \quad (\text{A.17c})$$

that is, we can change the order inside the scalar product. Moreover, inside of the scalar grade projection we have the commutative property

$$\langle ABC \rangle = \langle CAB \rangle = \langle BCA \rangle \quad (\text{A.17d})$$

As a particular case of (A.17b), we also have

$$A^\dagger * B = A * B^\dagger \quad (\text{A.17e})$$

The scalar product between two arbitrary multivectors can be expanded in terms of its graded components, that is

$$A * B \equiv \sum_{i=0}^n \langle A \rangle_i * B \equiv \langle A \rangle \langle B \rangle + \sum_{i=1}^n \langle A \rangle_i \cdot \langle B \rangle_i \quad (\text{A.17f})$$

The inner product between two k -vectors A_k and B_k , for $k \neq 0$ can be expressed with the scalar product,

$$A_k \cdot B_k = \langle A_k B_k \rangle = A_k * B_k \quad (\text{A.18})$$

The scalar product of two multivectors of different grade is zero, that is,

$$\langle \langle A \rangle_k \langle B \rangle_r \rangle = \langle A \rangle_k * \langle B \rangle_r = 0 \quad \text{if } r \neq k \quad (\text{A.19})$$

We call a multivector X “simple” when it satisfies

$$XX^\dagger = \langle XX^\dagger \rangle \quad (\text{A.20})$$

For simple multivectors, the inverse can be computed as

$$X^{-1} = \frac{X^\dagger}{XX^\dagger} = \frac{X^\dagger}{\langle XX^\dagger \rangle} \quad (\text{A.21})$$

when $XX^\dagger \neq 0$.

The norm of a multivector X is defined as the absolute value of the scalar product of X with its reverse, that is

$$|X|^2 = |\langle XX^\dagger \rangle| \quad (\text{A.22})$$

The absolute value in the right hand side of (A.22) is important since in pseudo-Euclidean spaces, the scalar product $\langle XX^\dagger \rangle$ can have negative values.

To define orthogonal transformations, we will use the following norm squared definition of a multivector as

$$\|X\|^2 = \langle XX^\dagger \rangle \quad (\text{A.23})$$

In contrast to (A.22), this norm can be negative.

Definition 1. We call an element A_k a k -blade if it is a k -vector which can be written as the product of k anticommuting vectors $a_1, a_2, \dots, a_k \in \mathcal{A}_{p,q}$, that is

$$A_k = a_1 a_2 \cdots a_k, \quad (\text{A.24a})$$

where

$$a_j a_i = -a_i a_j \quad (\text{A.24b})$$

for $i, j = 1, 2, \dots, k$, and $j \neq i$. Furthermore, a k -blade is simple thus it squares to a scalar quantity, that is

$$A_k^2 \equiv \langle A_k^2 \rangle = (-1)^{k(k-1)} \langle A_k A_k^\dagger \rangle = (-1)^{k(k-1)} \|A_k\|^2 \quad (\text{A.24c})$$

we can always write a k -blade as the wedge product of k linearly independent vectors b_k then

$$A_k = b_1 \wedge b_2 \wedge \cdots \wedge b_k \quad (\text{A.24d})$$

Note how we used (A.13) to write A_k^2 in terms of the norm squared of A_k .

We can express the inner and outer product between a vector a and a k -vector A_k as

$$a \cdot A_k = \frac{1}{2}(a A_k - (-1)^k A_k a) \quad (\text{A.25a})$$

$$a \wedge A_k = \frac{1}{2}(a A_k + (-1)^k A_k a) \quad (\text{A.25b})$$

We can also write the geometric product with respect to the inner and outer product, concretely let A be a multivector and a a vector, then

$$Aa = A \cdot a + A \wedge a \quad (\text{A.25c})$$

$$aA = a \cdot A + a \wedge A \quad (\text{A.25d})$$

The outer and the inner product satisfy the reordering rules

$$\mathbf{A}_i \cdot \mathbf{B}_j = (-1)^{i(j-1)} \mathbf{B}_j \cdot \mathbf{A}_i \quad (\text{A.26a})$$

$$\mathbf{A}_i \wedge \mathbf{B}_j = (-1)^{ij} \mathbf{B}_j \wedge \mathbf{A}_i \quad (\text{A.26b})$$

Projections to flat spaces can be defined via an inner product, we define a projection to an invertible s -blade $\mathbf{A}$ as

$$P_{\mathbf{A}}(\mathbf{X}_k) = \begin{cases} \mathbf{X}_k & \text{if } k = 0 \\ \mathbf{X}_k \cdot \mathbf{A} \mathbf{A}^{-1} & \text{if } 0 < k \leq s \\ 0 & \text{otherwise} \end{cases} \quad (\text{A.27})$$

projections satisfy the outermorphism property

$$P_{\mathbf{A}}(\mathbf{X} \wedge \mathbf{Y}) = P_{\mathbf{A}}(\mathbf{X}) \wedge P_{\mathbf{A}}(\mathbf{Y}) \quad (\text{A.28})$$

See proof in [Hestenes and Sobczyk \(1984\)](#) (Section 1.2). **The Unit Pseudoscalar** $\mathbf{I} \in \mathcal{G}_{p,q}^{p+q}$ of a geometric algebra $\mathcal{G}_{p,q}$ is the element of greatest grade that has magnitude one, $|\mathbf{I}| = 1$. Concretely, we can define it from the basis vectors

$$\mathbf{I} = \mathbf{e}_1 \mathbf{e}_2 \cdots \mathbf{e}_n = \mathbf{e}_1 \wedge \mathbf{e}_2 \wedge \cdots \wedge \mathbf{e}_n \quad (\text{A.29a})$$

$$\mathbf{I} \equiv \langle \mathbf{I} \rangle_n \equiv \langle \mathbf{I} \rangle_{p+q}, \quad |\mathbf{I}| = 1 \quad (\text{A.29b})$$

A.1. Identities

In this section we provide short proofs for some identities used. Consider the relations (A.25), and the definition (A.23). Let $\mathbf{A} \equiv \sum_i \mathbf{A}_i \equiv \sum_i \langle \mathbf{A}_i \rangle_i$ and $\mathbf{a} \equiv \langle \mathbf{a} \rangle_1$, now we show that

$$\begin{aligned} \|\mathbf{a} \cdot \mathbf{A}\|^2 &= \sum_i \|\mathbf{a} \cdot \mathbf{A}_i\|^2 = \sum_i \frac{1}{4} \|\mathbf{a} \mathbf{A}_i - (-1)^i \mathbf{A}_i \mathbf{a}\|^2 = \sum_i \frac{1}{4} (\mathbf{a} \mathbf{A}_i - (-1)^i \mathbf{A}_i \mathbf{a}) (\mathbf{A}_i^\dagger \mathbf{a} - (-1)^i \mathbf{a} \mathbf{A}_i^\dagger) \\ &= \sum_i \frac{1}{2} \mathbf{a}^2 \|\mathbf{A}_i\|^2 - \frac{1}{2} (-1)^i \langle \mathbf{A}_i \mathbf{a} \mathbf{A}_i^\dagger \mathbf{a} \rangle = \frac{1}{2} \mathbf{a}^2 \|\mathbf{A}\| - \frac{1}{2} \sum_i (-1)^i \langle \mathbf{A}_i \mathbf{a} \mathbf{A}_i^\dagger \mathbf{a} \rangle \end{aligned} \quad (\text{A.30a})$$

$$\begin{aligned} \|\mathbf{a} \wedge \mathbf{A}\|^2 &= \sum_i \|\mathbf{a} \wedge \mathbf{A}_i\|^2 = \sum_i \frac{1}{4} \|\mathbf{a} \mathbf{A}_i + (-1)^i \mathbf{A}_i \mathbf{a}\|^2 = \sum_i \frac{1}{4} (\mathbf{a} \mathbf{A}_i + (-1)^i \mathbf{A}_i \mathbf{a}) (\mathbf{A}_i^\dagger \mathbf{a} + (-1)^i \mathbf{a} \mathbf{A}_i^\dagger) \\ &= \sum_i \frac{1}{2} \mathbf{a}^2 \|\mathbf{A}_i\|^2 + \frac{1}{2} (-1)^i \langle \mathbf{A}_i \mathbf{a} \mathbf{A}_i^\dagger \mathbf{a} \rangle = \frac{1}{2} \mathbf{a}^2 \|\mathbf{A}\| + \frac{1}{2} \sum_i (-1)^i \langle \mathbf{A}_i \mathbf{a} \mathbf{A}_i^\dagger \mathbf{a} \rangle \end{aligned} \quad (\text{A.30b})$$

By which it is straightforward to show that

$$\|\mathbf{a} \wedge \mathbf{A}\|^2 + \|\mathbf{a} \cdot \mathbf{A}\|^2 = \|\mathbf{a}\mathbf{A}\|^2 = \mathbf{a}^2 \|\mathbf{A}\|^2 \quad (\text{A.30c})$$

Let $\mathbf{A}_k$ be a k -vector, $\mathbf{B}_{k-1}$ a $(k-1)$ -vector and $\mathbf{B}_{k+1}$ a $(k+1)$ -vector then using (A.13) and (A.25a), we obtain the following equalities

$$\begin{aligned} \langle \mathbf{a} \cdot \mathbf{A}_k \mathbf{B}_{k-1} \rangle &= \frac{1}{2} \langle \mathbf{a} \mathbf{A}_k \mathbf{B}_{k-1} \rangle - \frac{1}{2} (-1)^k \langle \mathbf{A}_k \mathbf{a} \mathbf{B}_{k-1} \rangle \\ &= \frac{1}{2} \langle \mathbf{a} \mathbf{A}_k \mathbf{B}_{k-1} \rangle - \frac{1}{2} (-1)^k \langle \mathbf{A}_k^\dagger \mathbf{B}_{k-1}^\dagger \mathbf{a} \rangle \\ &= \langle \mathbf{a} \mathbf{A}_k \mathbf{B}_{k-1} \rangle = \mathbf{a} \cdot \langle \mathbf{A}_k \mathbf{B}_{k-1} \rangle_1 \end{aligned} \quad (\text{A.31a})$$

$$\begin{aligned} \langle \mathbf{a} \wedge \mathbf{A}_k \mathbf{B}_{k-1} \rangle &= \frac{1}{2} \langle \mathbf{a} \mathbf{A}_k \mathbf{B}_{k+1} \rangle + \frac{1}{2} (-1)^k \langle \mathbf{A}_k \mathbf{a} \mathbf{B}_{k+1} \rangle \\ &= \frac{1}{2} \langle \mathbf{a} \mathbf{A}_k \mathbf{B}_{k+1} \rangle + \frac{1}{2} (-1)^k \langle \mathbf{A}_k^\dagger \mathbf{B}_{k+1}^\dagger \mathbf{a} \rangle \\ &= \langle \mathbf{a} \mathbf{A}_k \mathbf{B}_{k+1} \rangle = \mathbf{a} \cdot \langle \mathbf{A}_k \mathbf{B}_{k+1} \rangle_1 \end{aligned} \quad (\text{A.31b})$$

Let $\mathbf{A} \equiv \langle \mathbf{A} \rangle_r$ be an invertible r -blade, then

$$\mathbf{x} \cdot \mathbf{A} \mathbf{A}^{-1} = \frac{1}{2} (\mathbf{x} \mathbf{A} - (-1)^r \mathbf{A} \mathbf{x}) \mathbf{A}^{-1} = \frac{1}{2} (\mathbf{x} \mathbf{A} \mathbf{A}^{-1} - (-1)^r \mathbf{A} \mathbf{x} \mathbf{A}^{-1}) = \frac{1}{2} \mathbf{x} - \frac{1}{2} (-1)^r \mathbf{A} \mathbf{x} \mathbf{A}^{-1} \quad (\text{A.32})$$

Any vector $\mathbf{x}$ can be expressed as the component in $\mathbf{A}$ and as the components orthogonal to $\mathbf{A}$, having

$$\mathbf{x} = \mathbf{x} \mathbf{A} \mathbf{A}^{-1} = \mathbf{x} \cdot \mathbf{A} \mathbf{A}^{-1} + \mathbf{x} \wedge \mathbf{A} \mathbf{A}^{-1} = P_A(\mathbf{x}) + P_A^\perp(\mathbf{x}) \quad (\text{A.33})$$

where $P_A(\mathbf{x}) = \mathbf{x} \cdot \mathbf{A} \mathbf{A}^{-1}$ and $P_A^\perp(\mathbf{x}) = \mathbf{x} \wedge \mathbf{A} \mathbf{A}^{-1}$.

Let $\mathbf{X}$ and $\mathbf{Y}$ be general multivectors, then we have

$$P_A(\mathbf{X}\mathbf{Y}) = \mathbf{X} P_A(\mathbf{Y}), \quad \text{if } \mathbf{X} = P_A(\mathbf{X}) \quad (\text{A.34})$$

B. Differentiation by Multivectors

The geometric derivative can be defined with respect to the directional derivative as

$$\partial = \sum_{i=1}^n \mathbf{a}^i \mathbf{a}_i \cdot \partial \quad (\text{B.1a})$$

where the vectors $\mathbf{a}_i$ are any set of vectors that span all $\mathcal{A}_{p,q}$, and satisfy the following reciprocity relation

$$\mathbf{a}_i \cdot \mathbf{a}^j = \delta_{ij} \quad (\text{B.1b})$$

for $i, j = 1, 2, \dots, n$. Where

$$\delta_{ij} = \begin{cases} 1 & \text{if } i \neq j \\ 0 & \text{otherwise} \end{cases} \quad (\text{B.1c})$$

and where we define the directional derivative via the limit definition

$$\mathbf{a} \cdot \partial F(\mathbf{x}) = \lim_{\tau \rightarrow 0} \frac{F(\mathbf{x} + \tau \mathbf{a}) - F(\mathbf{x})}{\tau} \quad (\text{B.1d})$$

The chain rule succinctly satisfies

$$\partial_{\mathbf{x}} f(g(\mathbf{x})) = \bar{g}(\partial_{\mathbf{x}'} f(\mathbf{x}')) \quad (\text{B.2a})$$

where

$$\bar{g}(\mathbf{a}) = \partial_{\mathbf{x}} g(\mathbf{x}) \cdot \mathbf{a} \quad (\text{B.2b})$$

and

$$\bar{g}(\partial_{\mathbf{x}'}) = \partial_{\mathbf{x}} g(\mathbf{x}) \cdot \partial_{\mathbf{x}'} \quad (\text{B.2c})$$

Differentiation with respect to multivectors has to be defined via a multivector basis then

$$\partial_X = \sum_J \mathbf{e}^J \mathbf{e}_J * \partial_X \quad (\text{B.3})$$

where $\mathbf{e}_J$ is the basis for the geometric algebra $\mathcal{G}_{p,q}$ defined in (A.5). The directional derivative $\mathbf{A} * \partial_X$, which is a generalization of (B.1d), is defined with respect to a limit, having

$$\mathbf{A} * \partial F(\mathbf{X}) = \lim_{\tau \rightarrow 0} \frac{F(\mathbf{A} + \tau \mathbf{A}) - F(\mathbf{A})}{\tau} \quad (\text{B.4})$$

Differentiating scalar functions of multivector variable can be employed by understanding some properties of the derivative

$$\dot{\partial}_X \langle \mathbf{A}(X) \dot{\mathbf{X}} \rangle = \dot{\partial}_X \langle \dot{\mathbf{X}} \mathbf{A}(X) \rangle = \dot{\partial}_X \dot{\mathbf{X}} * \mathbf{A}(X) = \mathbf{A}(X) \quad (\text{B.5a})$$

$$\partial_X \phi(\mathbf{X}^\dagger) = \partial_{X^\dagger} \phi(\mathbf{X}) = \partial_X^\dagger \phi(\mathbf{X}) = (\partial_X \phi(\mathbf{X}))^\dagger \quad (\text{B.5b})$$

$$\partial_X \phi(\langle \mathbf{X} \rangle_k) = \partial_{\langle \mathbf{X} \rangle_k} \phi(\mathbf{X}) = \langle \partial_X \rangle_k \phi(\mathbf{X}) = \langle \partial_X \phi(\mathbf{X}) \rangle_k \quad (\text{B.5c})$$

$$\partial_X \phi(P(\mathbf{X})) = P(\partial_X) \phi(\mathbf{X}) = P(\partial_X \phi(\mathbf{X})) = P(\partial_X \phi(P(\mathbf{X}))), \quad \text{if } \mathbf{X} = P(\mathbf{X}) \quad (\text{B.5d})$$

$$\partial_X \langle f(\mathbf{X}) g(\mathbf{X}) \rangle_k = \dot{\partial}_X \langle \dot{f}(\mathbf{X}) g(\mathbf{X}) \rangle_k + \dot{\partial}_X \langle f(\mathbf{X}) \dot{g}(\mathbf{X}) \rangle_k \quad (\text{B.5e})$$

$$\partial_X \langle \mathbf{X} \mathbf{A} \mathbf{X}^\dagger \mathbf{B} \rangle = \dot{\partial}_X \langle \dot{\mathbf{X}} \mathbf{A} \mathbf{X}^\dagger \mathbf{B} \rangle + \dot{\partial}_X \langle \mathbf{X} \mathbf{A} \dot{\mathbf{X}}^\dagger \mathbf{B} \rangle = \mathbf{A} \mathbf{X}^\dagger \mathbf{B} + \mathbf{A}^\dagger \mathbf{X}^\dagger \mathbf{B}^\dagger \quad (\text{B.5f})$$

where the dot indicates the element in the product which is to be differentiated. Some directional derivatives of multivectors can be succinctly expressed as

$$\mathbf{Y} * \partial_X \mathbf{A} \mathbf{X} \mathbf{B} = \mathbf{A} \mathbf{Y} * \partial_X \mathbf{X} \mathbf{B} = \mathbf{A} \mathbf{Y} \mathbf{B} \quad (\text{B.6})$$

C. Conformal Geometric Algebra

Conformal Geometric Algebra is the extension of a vector in $\mathcal{A}_{p,q}$ space by adding two extra dimensions. To understand how points in $\mathcal{A}_{p,q}$ relate with points in $\mathcal{A}_{p+1,q+1}$, we define the conformal mapping:

Definition 2. *The conformal mapping $c : \mathcal{A}_{p,q} \mapsto \mathcal{A}_{p+1,q+1}$ is defined as the operation that takes a point in $\mathcal{A}_{p,q}$ and transforms it as:*

$$c(\mathbf{x}) = \mathbf{e}_o + \mathbf{x} + \frac{1}{2}\mathbf{x}^2\mathbf{e}_\infty \quad (\text{C.1a})$$

To define $\mathbf{e}_o$ and $\mathbf{e}_\infty$ we first introduce the extra two dimensions via two orthogonal vectors $\mathbf{e}_+$ and $\mathbf{e}_-$ that satisfy $\mathbf{e}_+^2 = 1$, $\mathbf{e}_-^2 = -1$, $\mathbf{e}_+ \cdot \mathbf{e}_- = 0$ and $\mathbf{e}_i \cdot \mathbf{e}_+ = \mathbf{e}_i \cdot \mathbf{e}_- = 0$ for $i = 1, 2, \dots, p+q$ then

$$\mathbf{e}_o = \frac{\mathbf{e}_- + \mathbf{e}_+}{\sqrt{2}}, \quad \mathbf{e}_\infty = \frac{\mathbf{e}_- - \mathbf{e}_+}{\sqrt{2}} \quad (\text{C.1b})$$

The inner product of two points $\mathbf{x}, \mathbf{y} \in \mathcal{A}_{p,q}$ extended through the conformal mapping (C.1a) relates to their Euclidean distance as

$$c(\mathbf{x}) \cdot c(\mathbf{y}) = -\frac{1}{2}\|\mathbf{x} - \mathbf{y}\|^2 \quad (\text{C.2a})$$

Thus, it is clear that the conformal mapping (C.1a) maps a point $\mathbf{x} \in \mathcal{A}_{p,q}$ to a null point in $\mathcal{A}_{p+1,q+1}$, that is

$$\|c(\mathbf{x})\|^2 = 0 \quad (\text{C.2b})$$

Note that the distance between the two conformal points $c(\mathbf{x})$ and $c(\mathbf{y})$ is equal to the Euclidean distance between $\mathbf{x}$ and $\mathbf{y}$, that is

$$\|c(\mathbf{x}) - c(\mathbf{y})\|^2 = \|c(\mathbf{x})\|^2 + \|c(\mathbf{y})\|^2 - 2c(\mathbf{x}) \cdot c(\mathbf{y}) = \|\mathbf{x} - \mathbf{y}\|^2 \quad (\text{C.2c})$$

We will denote the pseudoscalar of $\mathcal{G}_{p,q}$ by $\mathbf{I}$ and the pseudoscalar of $\mathcal{G}_{p+1,q+1}$ by $\mathbf{i}$. They relate via

$$\mathbf{i} \equiv \mathbf{I}\mathbf{e}_o \wedge \mathbf{e}_\infty = \mathbf{I}\mathbf{e}_+\mathbf{e}_- \quad (\text{C.3})$$

In conformal geometric algebra translations are expressed via the rotor $\mathbf{T} = 1 + \frac{1}{2}\mathbf{e}_\infty \mathbf{t}$. The translation can be written as

$$\mathbf{T}(\mathbf{x}) = \mathbf{T}\mathbf{x}\mathbf{T}^\dagger \quad (\text{C.4a})$$

We call $\mathbf{T}$ a translator. The translator $\mathbf{T}$ is unitary since it satisfies $\mathbf{T}\mathbf{T}^\dagger = 1$. A translation applied to an extended point (C.1a) yields

$$\mathbf{T}(c(\mathbf{x})) = \mathbf{T}c(\mathbf{x})\mathbf{T}^\dagger = c(\mathbf{x} + \mathbf{t}) \quad (\text{C.4b})$$

Rotation R in the extended conformal space can be expressed exactly in the same manner as in the Euclidean space since

$$R(c(x)) = \mathbf{R}c(x)\mathbf{R} = c(\mathbf{R}x\mathbf{R}^\dagger) = c(R(x)) \quad (\text{C.4c})$$

The coefficients $\mathcal{P}_1, \mathcal{P}_2, \dots, \mathcal{P}_4 \in G_{p,q}$ of $\mathbf{P} \in \mathcal{G}_{p+1,q+1}$ can be understood by expressing the multivector $\mathbf{P}$ via the basis vectors $\mathbf{e}_\infty$ and $\mathbf{e}_o$ as

$$\begin{aligned} \mathbf{P} &\equiv \mathbf{e}_o \mathcal{P}_1 + \mathbf{e}_\infty \mathcal{P}_2 + \mathbf{e}_o \wedge \mathbf{e}_\infty \mathcal{P}_3 + \mathcal{P}_4 \\ &= \mathbf{e}_o \wedge \mathcal{P}_1 + \mathbf{e}_\infty \wedge \mathcal{P}_2 + \mathbf{e}_o \wedge \mathbf{e}_\infty \wedge \mathcal{P}_3 + \mathcal{P}_4 \end{aligned} \quad (\text{C.5})$$

To determine the coefficients of a multivector $\mathbf{X} \in \mathcal{G}_{p+1,q+1}$ we can use the following set of operations

$$\mathcal{C}_1(\mathbf{X}) \equiv -P_I(\mathbf{e}_\infty \cdot \mathbf{X}) \quad (\text{C.6a})$$

$$\mathcal{C}_2(\mathbf{X}) \equiv -P_I(\mathbf{e}_o \cdot \mathbf{X}) \quad (\text{C.6b})$$

$$\mathcal{C}_3(\mathbf{X}) \equiv P_I((\mathbf{e}_o \wedge \mathbf{e}_\infty) \cdot \mathbf{X}) \quad (\text{C.6c})$$

$$\mathcal{C}_4(\mathbf{X}) \equiv P_I(\mathbf{X}) \quad (\text{C.6d})$$

$\mathcal{C}_i(\mathbf{X})$ extracts the i -th coefficient of the multivector $\mathbf{X}$. Note that P_I is given by (A.27) with $\mathbf{I}$ the unit pseudoscalar of $\mathcal{G}_{p,q}$.

Definition 3. The distance d^2 between two multivectors $\mathbf{P}$ and $\mathbf{Q}$ in $\mathcal{G}_{p+1,q+1}$ is defined as

$$d^2(\mathbf{P}, \mathbf{Q}) \equiv \|\mathcal{C}_3(\mathbf{P}) - \mathcal{C}_3(\mathbf{Q})\|^2 + \|\mathcal{C}_4(\mathbf{P}) - \mathcal{C}_4(\mathbf{Q})\|^2 \quad (\text{C.7})$$

where $\mathcal{C}_i(\mathbf{X})$ is defined by (C.6)

Proof of Theorem 3.2. We start by demonstrating how different components are affected by translations T defined via (C.4a). Consider $\mathbf{A} \in \mathcal{G}_{p,q}$ then the following

holds

$$\begin{aligned}
T(\mathbf{e}_o) &= (1 + \frac{1}{2}\mathbf{e}_\infty \mathbf{t})\mathbf{e}_o(1 - \frac{1}{2}\mathbf{e}_\infty \mathbf{t}) = \mathbf{e}_o - \frac{1}{4}\mathbf{e}_\infty \mathbf{t} \mathbf{e}_o \mathbf{e}_\infty \mathbf{t} + \frac{1}{2}\mathbf{e}_\infty \mathbf{t} \mathbf{e}_o - \frac{1}{2}\mathbf{e}_o \mathbf{e}_\infty \mathbf{t} \\
&= \mathbf{e}_o - \frac{1}{4}\mathbf{e}_\infty \mathbf{e}_o \mathbf{e}_\infty \mathbf{t}^2 - \frac{1}{2}(\mathbf{e}_\infty \mathbf{e}_o + \mathbf{e}_o \mathbf{e}_\infty) \mathbf{t} \\
&= \mathbf{e}_o + \frac{1}{2}\mathbf{t}^2 \mathbf{e}_\infty - \mathbf{e}_\infty \cdot \mathbf{e}_o \mathbf{t} \\
&= \mathbf{e}_o + \mathbf{t} + \frac{1}{2}\mathbf{t}^2 \mathbf{e}_\infty
\end{aligned} \tag{C.8a}$$

$$\begin{aligned}
T(\mathbf{e}_\infty) &= \mathbf{e}_\infty + \frac{1}{2}\mathbf{e}_\infty \mathbf{t} \mathbf{e}_\infty - \frac{1}{2}\mathbf{e}_\infty \mathbf{e}_\infty \mathbf{t} - \frac{1}{4}\mathbf{e}_\infty \mathbf{t} \mathbf{e}_\infty \mathbf{e}_\infty \mathbf{t} \\
&= \mathbf{e}_\infty - \frac{1}{2}\mathbf{e}_\infty^2 \mathbf{t} - \frac{1}{2}\mathbf{e}_\infty^2 - \frac{1}{4}\mathbf{e}_\infty \mathbf{t}^2 \mathbf{e}_\infty^2 = \mathbf{e}_\infty
\end{aligned} \tag{C.8b}$$

$$\begin{aligned}
T(\mathbf{A}) &= \mathbf{A} - \frac{1}{4}\mathbf{e}_\infty \mathbf{t} \mathbf{A} \mathbf{e}_\infty \mathbf{t} + \frac{1}{2}\mathbf{e}_\infty \mathbf{t} \mathbf{A} - \frac{1}{2}\mathbf{A} \mathbf{e}_\infty \mathbf{t} \\
&= \mathbf{A} + \frac{1}{2} \sum_k (\mathbf{e}_\infty \mathbf{t} \langle \mathbf{A} \rangle_k - \langle \mathbf{A} \rangle_k \mathbf{e}_\infty \mathbf{t}) \\
&= \mathbf{A} + \frac{1}{2} \sum_k (\mathbf{e}_\infty \mathbf{t} \langle \mathbf{A} \rangle_k - (-1)^k \mathbf{e}_\infty \langle \mathbf{A} \rangle_k \mathbf{t}) \\
&= \mathbf{A} + \mathbf{e}_\infty \sum_k \frac{1}{2} (\mathbf{t} \langle \mathbf{A} \rangle_k - (-1)^k \langle \mathbf{A} \rangle_k \mathbf{t}) \\
&= \mathbf{A} + \mathbf{e}_\infty \sum_k \mathbf{t} \cdot \langle \mathbf{A} \rangle_k = \mathbf{A} + \mathbf{e}_\infty \mathbf{t} \cdot \mathbf{A}
\end{aligned} \tag{C.8c}$$

where we used $\mathbf{e}_\infty \mathbf{e}_o \mathbf{e}_\infty = -2\mathbf{e}_\infty$ and that $\mathbf{e}_\infty^2 = 0$ and the reordering rule for the wedge product (A.26b) to find that $\mathbf{e}_\infty \langle \mathbf{A} \rangle_k = (-1)^k \langle \mathbf{A} \rangle_k \mathbf{e}_\infty$. In the last two steps of (C.8c) we used (A.25a) and (A.8) to recover the inner product of $\mathbf{t}$ with $\mathbf{A}$.

Now, applying a translation to (C.5) gives

$$\begin{aligned}
T(\mathbf{P}) &= T(\mathbf{e}_o) \wedge T(\mathcal{P}_1) + T(\mathbf{e}_\infty) \wedge T(\mathcal{P}_2) + T(\mathbf{e}_o) \wedge T(\mathbf{e}_\infty) \wedge T(\mathcal{P}_3) + T(\mathcal{P}_4) \\
&= (\mathbf{e}_o + \mathbf{t} + \frac{1}{2}\mathbf{t}^2 \mathbf{e}_\infty) \wedge (\mathcal{P}_1 + \mathbf{e}_\infty \mathbf{t} \cdot \mathcal{P}_1) + \mathbf{e}_\infty \wedge (\mathcal{P}_2 + \mathbf{e}_\infty \mathbf{t} \cdot \mathcal{P}_2) + \\
&\quad + (\mathbf{e}_o + \mathbf{t} + \frac{1}{2}\mathbf{t}^2 \mathbf{e}_\infty) \wedge \mathbf{e}_\infty \wedge (\mathcal{P}_3 + \mathbf{e}_\infty \mathbf{t} \cdot \mathcal{P}_3) + \mathcal{P}_4 + \mathbf{e}_\infty \mathbf{t} \cdot \mathcal{P}_4 \\
&= (\mathbf{e}_o + \mathbf{t} + \frac{1}{2}\mathbf{t}^2 \mathbf{e}_\infty) \wedge (\mathcal{P}_1 + \mathbf{e}_\infty \mathbf{t} \cdot \mathcal{P}_1) + \mathbf{e}_\infty \wedge \mathcal{P}_2 + (\mathbf{e}_o + \mathbf{t}) \wedge \mathbf{e}_\infty \wedge \mathcal{P}_3 + \mathcal{P}_4 + \mathbf{e}_\infty \mathbf{t} \cdot \mathcal{P}_4
\end{aligned} \tag{C.9}$$

which putting in the same form as (C.5) we have

$$T(\mathbf{P}) = \mathbf{e}_o \mathcal{P}_1 + \mathbf{e}_\infty \left(\frac{1}{2} \mathbf{t}^2 \mathcal{P}_1 - \mathbf{t} \wedge (\mathbf{t} \cdot \mathcal{P}_1) + \mathcal{P}_2 + \mathbf{t} \cdot \mathcal{P}_4 - \mathbf{t} \wedge \mathcal{P}_3 \right) + \mathbf{e}_o \wedge \mathbf{e}_\infty (\mathbf{t} \cdot \mathcal{P}_1 + \mathcal{P}_3) + \mathbf{t} \wedge \mathcal{P}_1 + \mathcal{P}_4 \tag{C.10a}$$

The point at infinity $\mathbf{e}_\infty$ and the point at the origin $\mathbf{e}_o$ commute with rotors in $\mathcal{G}_{p,q}$, that is, $R(\mathbf{e}_o) = \mathbf{e}_o$ and $R(\mathbf{e}_\infty) = \mathbf{e}_\infty$. Thus, applying a rotation R to (C.5) gives

$$\begin{aligned}
R(\mathbf{P}) &= R(\mathbf{e}_o) \wedge R(\mathcal{P}_1) + R(\mathbf{e}_\infty) \wedge R(\mathcal{P}_2) + R(\mathbf{e}_o) \wedge R(\mathbf{e}_\infty) \wedge R(\mathcal{P}_3) + R(\mathcal{P}_4) \\
&= \mathbf{e}_o \wedge R(\mathcal{P}_1) + \mathbf{e}_\infty \wedge R(\mathcal{P}_2) + \mathbf{e}_o \wedge \mathbf{e}_\infty \wedge R(\mathcal{P}_3) + R(\mathcal{P}_4)
\end{aligned} \tag{C.10b}$$

Thus, for the coefficients of rotations we have that

$$\mathcal{C}_i(R(\mathbf{P})) = R(\mathcal{C}_i(\mathbf{P})) = R(P_i), \text{ for } i = 1, 2, 3, 4 \quad (\text{C.10c})$$

Since $R(\mathcal{P}_i) \in G_{p,q}$ we can also use (C.10a) to compose a rotation with a translation. Replace $\mathbf{P}$ by $R(\mathbf{P})$ and $\mathcal{P}_i$ by $R(\mathcal{P}_i)$ in (C.10a). Then use (C.6) to extract the coefficients to arrive at (3.7a). For the inverse transformation $\bar{R}\bar{T}$ we replace $\mathbf{t}$ by $-\mathbf{t}$, T by $\bar{T}$ and $\mathbf{P}$ by $\mathbf{Q}$ in (C.10a) then apply $\bar{R}$ to that result using (C.10b) to finally obtain (3.7b). $\square$

Theorem 3.5. *The translation vector $\mathbf{t}$ can be determined exactly from (3.7a) as*

$$\begin{aligned} \mathbf{t} &= (\mathcal{Q}_3 + \mathcal{Q}_4 - \underline{R}(\mathcal{P}_3 + \mathcal{P}_4))\underline{R}(\mathcal{P}_1^{-1}) \\ &= (\mathcal{Q}_3 + \mathcal{Q}_4 - \underline{R}(\mathcal{P}_3 + \mathcal{P}_4))\mathcal{Q}_1^{-1} \end{aligned} \quad (3.12)$$

Proof of Theorem 3.5. Take the bottom two equations of (3.7a) and add them together, then

$$\mathcal{Q}_3 + \mathcal{Q}_4 = \mathbf{t} \cdot \underline{R}(\mathcal{P}_1) + \underline{R}(\mathcal{P}_3) + \mathbf{t} \wedge \underline{R}(\mathcal{P}_1) + \underline{R}(\mathcal{P}_4)$$

Using (A.25d) we have $\mathbf{t} \cdot \underline{R}(\mathcal{P}_1) + \mathbf{t} \wedge \underline{R}(\mathcal{P}_1) = \mathbf{t}\underline{R}(\mathcal{P}_1)$ and considering that $\mathcal{P}_1$ is a simple multivector, then solving for $\mathbf{t}$ we find that

$$\mathbf{t} = (\mathcal{Q}_3 + \mathcal{Q}_4 - \underline{R}(\mathcal{P}_3 + \mathcal{P}_4))\underline{R}(\mathcal{P}_1^{-1})$$

then using the first equation of (3.7a) we can put this in the form

$$\mathbf{t} = (\mathcal{Q}_3 + \mathcal{Q}_4 - \underline{R}(\mathcal{P}_3 + \mathcal{P}_4))\mathcal{Q}_1^{-1}$$

$\square$

C.1. Extending Euclidean points to CGA

Having described the general methods, which can be applied to multivector clouds in CGA, we also propose an approach that can deal with 3D points in Vanilla Geometric Algebra (VGA). Concretely, we consider two point clouds in VGA and extend it to CGA. We then express how the CGA vectors relate with the VGA vectors. We consider the noisy relationship between point clouds in VGA, then understand how that noisy relationship extends to CGA.

Given two point clouds $\mathbf{x}'_i \in \mathcal{A}_3$ and $\mathbf{y}'_i \in \mathcal{A}_3$, for $i = 1, 2, \dots, \ell$, we aim to find the rigid transformation that best aligns both point clouds consisting of a rotator $\mathbf{R} \in \mathcal{R}$ and a vector $\mathbf{t} \in \mathcal{A}_3$. Assuming

$$\mathbf{y}'_i = \mathbf{R}(\mathbf{x}'_{j_i}) + \mathbf{t} + \mathbf{n}_i \quad (\text{C.11})$$

where $\mathbf{n}_i$ is assumed to be a measurement noise modeled by a Gaussian distribution. Also, we assume that j_i is some unknown index vector which does not have repeated indices, making it not possible for one-to-many correspondences to exist. In this context of the alignment problem, we set $\mathbf{X}_i = \mathbf{x}_i$ and $\mathbf{Y}_i = \mathbf{y}_i$ in (3.2) then we can proceed via the method described in Section 3.2.

Definition 4. We define the conformal points $\mathbf{x}_i \in \mathcal{A}_{4,1}$ and $\mathbf{y}_i \in \mathcal{A}_{4,1}$ as the points extended from the Euclidean points $\mathbf{x}'_i \in \mathcal{A}_3$ and $\mathbf{y}'_i \in \mathcal{A}_3$ via the conformal mapping (C.1a), that is

$$\mathbf{x}_i = c(\mathbf{x}'_i) = \mathbf{e}_o + \mathbf{x}'_i + \frac{1}{2}(\mathbf{x}'_i)^2 \mathbf{e}_\infty \quad (\text{C.12a})$$

$$\mathbf{y}_i = c(\mathbf{y}'_i) = \mathbf{e}_o + \mathbf{y}'_i + \frac{1}{2}(\mathbf{y}'_i)^2 \mathbf{e}_\infty \quad (\text{C.12b})$$

Theorem C.1. When the Euclidean points relate via (C.11) then the conformal points (C.12) will relate via

$$\mathbf{y}_i = TR(\mathbf{x}_{j_i}) + \mathbf{r}_i \quad (\text{C.13a})$$

where $\mathbf{r}_i$ is the conformal noise given by

$$\mathbf{r}_i = \mathbf{n}_i + \frac{1}{2}(\mathbf{n}_i^2 + 2\mathbf{n}_i \cdot (R(\mathbf{x}'_{j_i}) + \mathbf{t})) \mathbf{e}_\infty \quad (\text{C.13b})$$

Furthermore, the noise from the view point of $\mathbf{x}_{j_i}$ is given by

$$\mathbf{w}_i = \bar{R}\bar{T}(\mathbf{r}_i) = \bar{R}(\mathbf{n}_i) + \frac{1}{2}(\mathbf{n}_i^2 + 2\bar{R}(\mathbf{n}_i) \cdot \mathbf{x}'_{j_i}) \mathbf{e}_\infty \quad (\text{C.14a})$$

and

$$\mathbf{x}_{j_i} = \bar{R}\bar{T}(\mathbf{y}_i) - \mathbf{w}_i \quad (\text{C.14b})$$

When $\mathbf{n}_i$ is assumed to be correlated Gaussian noise then the $\mathbf{w}_i$ has a Generalized chi-squared distribution. When $\mathbf{n}_i$ is Gaussian but uncorrelated then the probability distribution does not depend on the rotation R .

Proof. Recall (C.11) and (C.12) then

$$\begin{aligned} c(\mathbf{y}'_i) &= c(\mathbf{R}\mathbf{x}'_{j_i}\mathbf{R}^\dagger + \mathbf{t} + \mathbf{n}_i) \\ &= \mathbf{e}_o + \mathbf{R}\mathbf{x}'_{j_i}\mathbf{R}^\dagger + \mathbf{t} + \mathbf{n}_i + \frac{1}{2}(\mathbf{R}\mathbf{x}'_{j_i}\mathbf{R}^\dagger + \mathbf{t} + \mathbf{n}_i)^2 \mathbf{e}_\infty \\ &= \mathbf{e}_o + \mathbf{R}\mathbf{x}'_{j_i}\mathbf{R}^\dagger + \mathbf{t} + \frac{1}{2}(\mathbf{R}\mathbf{x}'_{j_i}\mathbf{R}^\dagger + \mathbf{t})^2 \mathbf{e}_\infty + \mathbf{n}_i + \mathbf{n}_i \cdot (\mathbf{R}\mathbf{x}'_{j_i}\mathbf{R}^\dagger + \mathbf{t}) \mathbf{e}_\infty + \frac{1}{2}\mathbf{n}_i^2 \mathbf{e}_\infty \\ &= c(\mathbf{R}\mathbf{x}'_{j_i}\mathbf{R}^\dagger + \mathbf{t}) + \mathbf{n}_i + \mathbf{n}_i \cdot (\mathbf{R}\mathbf{x}'_{j_i}\mathbf{R}^\dagger + \mathbf{t}) \mathbf{e}_\infty + \frac{1}{2}\mathbf{n}_i^2 \mathbf{e}_\infty \end{aligned} \quad (\text{C.15})$$

recalling how translations and rotations relate from the embedding space to the embedded space by (C.4), we can easily find that $c(\mathbf{R}\mathbf{x}'_{j_i}\mathbf{R}^\dagger + \mathbf{t}) = TRc(\mathbf{x}'_{j_i}) = TR(\mathbf{x}_{j_i})$. By also setting $\mathbf{r}_i = \mathbf{n}_i + \mathbf{n}_i \cdot (\mathbf{R}\mathbf{x}'_{j_i}\mathbf{R}^\dagger + \mathbf{t})\mathbf{e}_\infty + \frac{1}{2}\mathbf{n}_i^2\mathbf{e}_\infty$ we arrive at (C.13a). To show how (C.14a) holds we start by taking $\bar{T}$ to $\mathbf{r}_i$ thus

$$\begin{aligned}\bar{T}(\mathbf{r}_i) &= \bar{T}(\mathbf{n}_i) + \frac{1}{2}(\mathbf{n}_i^2 + 2\mathbf{n}_i \cdot (\mathbf{R}\mathbf{x}'_{j_i} + \mathbf{t}))\bar{T}(\mathbf{e}_\infty) \\ &= \mathbf{n}_i - \mathbf{t} \cdot \mathbf{n}_i\mathbf{e}_\infty + \frac{1}{2}(\mathbf{n}_i^2 + 2\mathbf{n}_i \cdot (\mathbf{R}\mathbf{x}'_{j_i} + \mathbf{t}))\mathbf{e}_\infty \\ &= \mathbf{n}_i + \frac{1}{2}(\mathbf{n}_i^2 + 2\mathbf{n}_i \cdot \mathbf{R}\mathbf{x}'_{j_i})\mathbf{e}_\infty \\ &= \mathbf{n}_i + \frac{1}{2}((\bar{R}(\mathbf{n}_i))^2 + 2\bar{R}(\mathbf{n}_i) \cdot \mathbf{x}'_{j_i})\mathbf{e}_\infty\end{aligned}\tag{C.16}$$

then applying a rotation $\bar{R}$ to the above result we obtain

$$\begin{aligned}\bar{R}\bar{T}(\mathbf{r}_i) &= \bar{R}(\mathbf{n}_i) + \frac{1}{2}((\bar{R}(\mathbf{n}_i))^2 + 2\bar{R}(\mathbf{n}_i) \cdot \mathbf{x}'_{j_i})\bar{R}(\mathbf{e}_\infty) \\ &= \bar{R}(\mathbf{n}_i) + \frac{1}{2}((\bar{R}(\mathbf{n}_i))^2 + 2\bar{R}(\mathbf{n}_i) \cdot \mathbf{x}'_{j_i})\mathbf{e}_\infty\end{aligned}\tag{C.17}$$

note how we can replace $\mathbf{z}_i = \bar{R}(\mathbf{n}_i)$ to get

$$\bar{R}\bar{T}(\mathbf{r}_i) = \mathbf{z}_i + \frac{1}{2}(\mathbf{z}_i^2 + 2\mathbf{z}_i \cdot \mathbf{x}'_{j_i})\mathbf{e}_\infty\tag{C.18}$$

If $\mathbf{n}_i$ is uncorrelated Gaussian distributed then $\mathbf{z}_i$ will have the same distribution as $\mathbf{n}_i$ that does not depend on the rotation R .

Now with respect to the probability distribution of $\mathbf{r}_i$ first note that the terms $\mathbf{n}_i$ and $\mathbf{n}_i \cdot (\mathbf{R}\mathbf{x}'_{j_i} + \mathbf{t})$ have a Gaussian probability distribution, while the term $\mathbf{n}_i^2$ has a noncentral chi-squared distribution.

Note that a generalized chi squared distribution is obtained by the sum of a random variable with a Gaussian probability distribution, that is the terms $\mathbf{n}_i$ and $\mathbf{n}_i \cdot (\mathbf{R}\mathbf{x}'_{j_i} + \mathbf{t})$, and a random variable with a noncentral chi-squared distribution, which means that the overall noise $\mathbf{r}_i$ must have a generalized chi-squared distribution. However the component of $\mathbf{r}_i$ in $\mathcal{A}_{p,q}$, that is $\mathbf{n}_i$, will continue to have a Gaussian probability distribution. $\square$

D. Multilinear Transformations

Definition 5. A multilinear transformation $F : \mathcal{G}_{p,q} \mapsto \mathcal{G}_{p,q}$ is a transformation which is linear in its arguments and maps multivectors into multivectors. F is

linear when it is equal to its own differential, then given the following definition for the differential $\underline{F}$ of F

$$\underline{F}(X) = X * \partial_Y F(Y) = X^\dagger * \partial_Y^\dagger F(Y) \quad (\text{D.1a})$$

We state that F is linear when

$$\underline{F}(X) \equiv F(X) \quad (\text{D.1b})$$

The adjoint of multilinear transformations follows a similar definition to the adjoint of linear transformations, it is defined as

$$\bar{F}(X) = \partial_Y^\dagger X^\dagger * \underline{F}(Y) \quad (\text{D.1c})$$

implying that

$$\underline{F}(X) * Y^\dagger = X^\dagger * \bar{F}(Y) \quad (\text{D.1d})$$

Using (D.1c) and the rules of the derivatives (B.6), we show how we obtain (D.1d)

$$\begin{aligned} \bar{F}(X) * Y^\dagger &= (\partial_Z^\dagger X^\dagger * \underline{F}(Z)) * Y^\dagger = \partial_Z^\dagger * Y^\dagger X^\dagger * \underline{F}(Z) \\ &= \partial_Z * Y X^\dagger * \underline{F}(Z) = X^\dagger * \underline{F}(Y * \partial_Z Z) \\ &= X^\dagger * \underline{F}(Y) \end{aligned} \quad (\text{D.2})$$

Theorem D.1. *The multilinear function*

$$F(X) = \sum_{k=1}^m A_k^\dagger X B_k + A_k X B_k^\dagger \quad (\text{D.3})$$

is symmetric for any $A_k, B_k \in \mathcal{G}_{p,q}$.

Proof. Considering the relation (D.1d), we then compute

$$\begin{aligned} F(X) * Y^\dagger &= \sum_{k=1}^m \langle A_k^\dagger X B_k Y^\dagger + A_k X B_k^\dagger Y^\dagger \rangle \\ &= \sum_{k=1}^m \langle (A_k^\dagger X B_k)^\dagger Y + (A_k X B_k^\dagger)^\dagger Y \rangle \\ &= \sum_{k=1}^m \langle B_k^\dagger X^\dagger A_k Y + B_k X^\dagger A_k^\dagger Y \rangle \\ &= \sum_{k=1}^m \langle X^\dagger A_k Y B_k^\dagger + X^\dagger A_k^\dagger Y B_k \rangle \\ &= X^\dagger * F(Y) \end{aligned} \quad (\text{D.4})$$

where we used (A.17e), (A.17d) and (A.11a). $\square$

Definition 6. An *eigenmultivector* X of a multilinear function F is a multivector that satisfies

$$F(X) = \lambda X \quad (\text{D.5})$$

where λ is a scalar.

A function F which can be expressed via its eigenmultivectors, has a special decomposition. In particular symmetric multilinear functions can always be expressed via the theorem that follows (see Theo. D.2). Note that symmetric multilinear functions satisfy $\bar{F}(X) = \underline{F}(X)$.

Theorem D.2. Let F be a symmetric multilinear function, let $\lambda_1, \lambda_2, \dots, \lambda_m$ be its unique eigenvalues, let $\mathbf{P}_1, \mathbf{P}_2, \dots, \mathbf{P}_m$ be its eigenmultivectors and let $\mathbf{P}^1, \mathbf{P}^2, \dots, \mathbf{P}^m$ be the reciprocal multivectors. Then, we can write the multilinear function F in the following form

$$F(X) = \sum_{k=1}^m \lambda_k \langle X \mathbf{P}^k \rangle \mathbf{P}_k \quad (\text{D.6a})$$

with $\langle \mathbf{P}_i \mathbf{P}^j \rangle = \delta_{ij}$. The $\mathbf{P}_i$'s satisfy the eigenvalue equation (D.5), that is

$$F(\mathbf{P}_i) = \lambda_i \mathbf{P}_i \quad (\text{D.6b})$$

Lemma D.1. Assume that the following relationship holds $G(\mathbf{Z}) = \underline{U} F \bar{U}(\mathbf{Z})$. Let F have the spectral decomposition as in Theorem D.2, then G has the spectral decomposition

$$G(\mathbf{Z}) = \sum_{k=1}^m \lambda_k \langle \mathbf{Z} \mathbf{Q}^k \rangle \mathbf{Q}_k \quad (\text{D.7a})$$

with

$$\mathbf{Q}_k = s_k \underline{U}(\mathbf{P}_k) = s_k \underline{U} \mathbf{P}_k \mathbf{U}^\dagger, \quad \mathbf{Q}^k = s_k^{-1} \underline{U}(\mathbf{P}^k) = s_k^{-1} \underline{U} \mathbf{P}^k \mathbf{U}^\dagger \quad (\text{D.7ba, D.7bb})$$

where $s_k \in \mathbb{R}$.

Proof. Given $G(\mathbf{Z}) = \underline{U} F \bar{U}(\mathbf{Z})$ and assuming that F can be expressed as in (D.6a) we have that

$$\begin{aligned} G(\mathbf{Z}) &= \sum_{k=1}^m \langle \mathbf{U}^\dagger \mathbf{Z} \mathbf{U} \mathbf{P}^k \rangle \underline{U} \mathbf{P}_k \mathbf{U}^\dagger \\ &= \sum_{k=1}^m \langle \mathbf{Z} \mathbf{U} \mathbf{P}^k \mathbf{U}^\dagger \rangle \underline{U} \mathbf{P}_k \mathbf{U}^\dagger \\ &= \sum_{k=1}^m \langle \mathbf{Z} \mathbf{Q}^k \rangle \mathbf{Q}_k \end{aligned} \quad (\text{D.3})$$

with $\mathbf{Q}_k = s_k \underline{U}(\mathbf{P}_k) = s_k \mathbf{U} \mathbf{P}_k \mathbf{U}^\dagger$ and $\mathbf{Q}^k = s_k^{-1} \underline{U}(\mathbf{P}^k) = \mathbf{U} \mathbf{P}^k \mathbf{U}^\dagger$, with $s_k \in \mathbb{R}$. To conclude we also show that the reciprocity between the eigenmultivectors $\mathbf{Q}_k$ still holds

$$\langle \mathbf{Q}_i \mathbf{Q}^j \rangle = \langle s_k \mathbf{U} \mathbf{P}_i \mathbf{U}^\dagger s_k^{-1} \mathbf{U} \mathbf{P}^j \mathbf{U}^\dagger \rangle = \langle \mathbf{P}_i \mathbf{P}^j \rangle = \delta_{ij} \quad (\text{D.4})$$

□

Corollary 3.1. *Under the assumptions of Theorem 3.1 the scalar in (3.4) can be determined as follows*

$$s_i = \langle \mathbf{Q}_i \mathbf{Q}_{\text{ref}} \rangle \langle \mathbf{P}_i \mathbf{P}_{\text{ref}} \rangle^{-1} \quad (\text{3.5})$$

where $\mathbf{P}_{\text{ref}}$ and $\mathbf{Q}_{\text{ref}}$ are both multivectors in $\mathcal{G}_{4,1}$ that are related as $\mathbf{Q}_{\text{ref}} = \underline{U}(\mathbf{P}_{\text{ref}})$.

Proof of Corollary 3.1. Assuming $\mathbf{Q}_i = s_i \underline{U}(\mathbf{P}_i)$ and $\mathbf{Q}_{\text{ref}} = \underline{U}(\mathbf{P}_{\text{ref}})$ then we have

$$\begin{aligned} \langle \mathbf{Q}_i \mathbf{Q}_{\text{ref}} \rangle &= \langle s_i \underline{U}(\mathbf{P}_i) \underline{U}(\mathbf{P}_{\text{ref}}) \rangle \\ &= s_i \langle \mathbf{U} \mathbf{P}_i \mathbf{U}^\dagger \mathbf{U} \mathbf{P}_{\text{ref}} \mathbf{U}^\dagger \rangle = s_i \langle \mathbf{P}_i \mathbf{P}_{\text{ref}} \rangle \end{aligned} \quad (\text{D.5})$$

where we used (A.17d) to reverse the order of the multivectors and also noting that $\mathbf{U} \mathbf{U}^\dagger = 1$. Then by solving (D.5) for s_i , gives $s_i = \langle \mathbf{Q}_i \mathbf{Q}_{\text{ref}} \rangle \langle \mathbf{P}_i \mathbf{P}_{\text{ref}} \rangle^{-1}$, as intended. □

Theorem D.3. *Assume that $\mathbf{X}_i$ and $\mathbf{Y}_i$ are vectors. Let*

$$\mathbf{P}_{\text{ref}} = (1 + \mathbf{i})(1 + \bar{\mathbf{X}}) \wedge \mathbf{e}_\infty \quad (\text{D.6a})$$

$$\mathbf{Q}_{\text{ref}} = (1 + \mathbf{i})(1 + \bar{\mathbf{Y}}) \wedge \mathbf{e}_\infty \quad (\text{D.6b})$$

where

$$\bar{\mathbf{X}} = \frac{1}{\ell} \sum_{i=1}^{\ell} \mathbf{X}_i, \quad \bar{\mathbf{Y}} = \frac{1}{\ell} \sum_{i=1}^{\ell} \mathbf{Y}_i \quad (\text{D.7a, D.7b})$$

and the multivectors $\mathbf{X}_i$ and $\mathbf{Y}_i$ satisfy the relationship (3.1), with $N_i = 0$ then

$$\mathbf{Q}_{\text{ref}} = \underline{U}(\mathbf{P}_{\text{ref}}) \quad (\text{D.8})$$

Proof. Considering (3.1), it is straightforward to show that

$$\bar{\mathbf{Y}} = \frac{1}{\ell} \sum_{i=1}^{\ell} \mathbf{Y}_i = \frac{1}{\ell} \sum_{i=1}^{\ell} \underline{U}(\mathbf{X}_i) = \frac{1}{\ell} \sum_{i=1}^{\ell} \underline{U}(\mathbf{X}_i) = \underline{U}(\bar{\mathbf{X}}) \quad (\text{D.9})$$

Recall that the point at infinity and the unit pseudoscalar of $\mathcal{G}_{p+1,q+1}$ are invariant to rotation and translations, that is, $\underline{U}(\mathbf{e}_\infty) = TR(\mathbf{e}_\infty) = \mathbf{e}_\infty$ and $\underline{U}(\mathbf{i}) = \mathbf{U} \mathbf{i} \mathbf{U}^\dagger =$

$i\mathbf{U}\mathbf{U}^\dagger = i$. Then by the outermorphism properties of orthogonal transformations we have

$$\begin{aligned}\mathbf{Q}_{\text{ref}} &= (1 + i)(1 + \bar{\mathbf{Y}}) \wedge \mathbf{e}_\infty = (\mathbf{U}\mathbf{U}^\dagger + \mathbf{U}i\mathbf{U}^\dagger)(\mathbf{U}\mathbf{U}^\dagger + \mathbf{U}\bar{\mathbf{X}}\mathbf{U}^\dagger) \wedge (\mathbf{U}\mathbf{e}_\infty\mathbf{U}^\dagger) \\ &= \mathbf{U}((1 + i)(1 + \bar{\mathbf{Y}}) \wedge \mathbf{e}_\infty) \mathbf{U}^\dagger = \underline{\mathbf{U}}(\mathbf{P}_{\text{ref}})\end{aligned}\quad (\text{D.10})$$

□

Definition 7 (Multilinear Space). *A multilinear space $\mathbb{S}$ is a linear space that spans some multivector basis. Let $\mathbf{A}_1, \mathbf{A}_2, \dots, \mathbf{A}_m \in \mathcal{G}_{p,q}$ be a basis for the multilinear space $\mathbb{S}$, then the projection to the multilinear space $\mathbb{S}$ can be expressed as*

$$P_{\mathbb{S}}(\mathbf{X}) = \sum_{k=1}^m \langle \mathbf{X} \mathbf{A}^k \rangle \mathbf{A}_k \quad (\text{D.11})$$

where $\mathbf{A}^k$ are the reciprocals of $\mathbf{A}_k$, that is, $\mathbf{A}_i * \mathbf{A}^j = \delta_{ij}$.

E. Solving the Optimal Translation and Rotation Problem

Lemma E.1. *The Lagrangian associated with the constraint $\mathbf{X}\mathbf{X}^\dagger = 1$ is given by*

$$\mathcal{L}(\mathbf{\Lambda}, \mathbf{X}) = \langle \mathbf{\Lambda}(\mathbf{X}\mathbf{X}^\dagger - 1) \rangle \quad (\text{E.1})$$

where $\mathbf{\Lambda} \in \mathcal{G}_{p,q}$ is the Lagrange multiplier.

Lemma E.2 (Multilinear Constraint). *Under the constraint $\mathbf{X} \in \mathbb{S}$, where $\mathbb{S}$ is a multilinear space defined in Definition 7, the stationary points of $J(\mathbf{X})$ satisfy the equation*

$$P_{\mathbb{S}} F P_{\mathbb{S}}(\mathbf{X}) = 0 \quad (\text{E.2a})$$

where $F(\mathbf{X}) = \partial_{\mathbf{X}} J(\mathbf{X})$. As a particular case of this theorem we also state that when $\mathbf{X}$ is restricted to some grades $K = (k_1, k_2, \dots, k_s)$ then the stationary points of J will be the solution to the equation

$$\langle F(\langle \mathbf{X} \rangle_K) \rangle_K = 0 \quad (\text{E.2b})$$

Theorem E.1. *Let $\mathbb{S}$ and $P_{\mathbb{S}}$ be defined by Definition 7. The equation associated with the optimization problem*

$$\begin{aligned}&\underset{\mathbf{X}}{\text{maximize}} && J(\mathbf{X}) \\ &\text{subject to} && \mathbf{X}\mathbf{X}^\dagger = \pm 1, \\ &&& \mathbf{X} \in \mathbb{S}\end{aligned}\quad (\text{E.3})$$

is given by the multilinear eigenvalue equation

$$P_{\mathbb{S}} F P_{\mathbb{S}}(\mathbf{X}) = P_{\mathbb{S}}((\mathbf{\Lambda} + \mathbf{\Lambda}^{\dagger}) P_{\mathbb{S}}(\mathbf{X})) \quad (\text{E.4a})$$

where $\mathbf{\Lambda}$ is the multivector Lagrange multiplier associated with the constraint $\mathbf{X}\mathbf{X}^{\dagger} = \pm 1$ and where $F(\mathbf{X}) = \partial_{\mathbf{X}}^{\dagger} J(\mathbf{X})$. Note that the solution is free to chose the sign $s = \pm 1$ for the constraint $\mathbf{X}\mathbf{X}^{\dagger} = s$. The constraint is necessary to restrict the quantity $\mathbf{X}\mathbf{X}^{\dagger}$ to be a non-vanishing scalar. If $\mathbf{X} = P_{\mathbb{S}}(\mathbf{X})$ then we may write

$$P_{\mathbb{S}} F(\mathbf{X}) = P_{\mathbb{S}}((\mathbf{\Lambda} + \mathbf{\Lambda}^{\dagger}) \mathbf{X}) \quad (\text{E.4b})$$

Theorem E.2. Let $p = 3$ and $q = 0$ then when $\mathbf{X}$ is restricted to live in $\mathcal{R}$, then the Lagrange multiplier $\mathbf{\Lambda}$ associated with the optimization problem

$$\underset{\mathbf{X}}{\text{maximize}} J(\mathbf{X}), \quad \text{subject to } \mathbf{X} \in \mathcal{R} \quad (\text{E.5})$$

can be just a scalar.

Proof. In $\mathcal{G}_3$ a rotator $\mathbf{X} \in \mathcal{R}$ belongs to the even subalgebra of $\mathcal{G}_3$, thus $\mathbf{X} \in \mathcal{G}_3^+ = \mathcal{G}_3^0 \oplus \mathcal{G}_3^2$, then we can write $\mathbf{X} = \alpha + \mathbf{B}$, where $\alpha \in \mathbb{R}$ and where $\mathbf{B} \in \mathcal{G}_3^2$ is a bivector. But since $\mathbf{B}$ lives in $\mathcal{G}_3$, then it is always a blade, being a blade then it squares to a scalar. Thus we write

$$\mathbf{X}\mathbf{X}^{\dagger} = (\alpha + \mathbf{B})(\alpha - \mathbf{B}) = \alpha^2 - \mathbf{B}^2 + \alpha\mathbf{B} - \alpha\mathbf{B} = \alpha^2 - \mathbf{B}^2 = \langle \alpha^2 - \mathbf{B}^2 \rangle \quad (\text{E.6})$$

since the quantity $\mathbf{X}\mathbf{X}^{\dagger}$ is a scalar then writing $\mathbf{\Lambda}$ in its different grades parts $\mathbf{\Lambda} = \mathbf{\Lambda}_0 + \mathbf{\Lambda}_1 + \mathbf{\Lambda}_2 + \mathbf{\Lambda}_3$ and considering (A.19) we have

$$\langle \mathbf{\Lambda} \mathbf{X} \mathbf{X}^{\dagger} \rangle = \sum_{k=0}^3 \langle \mathbf{\Lambda}_k \mathbf{X} \mathbf{X}^{\dagger} \rangle = \langle \mathbf{\Lambda}_0 \mathbf{X} \mathbf{X}^{\dagger} \rangle \quad (\text{E.7})$$

thus it is always valid that $\mathbf{\Lambda} = \langle \mathbf{\Lambda} \rangle = \mathbf{\Lambda}_0$ □

By Theorem E.2 for a rotor $\mathbf{R} \in \mathcal{G}_3$ the Lagrangian associated with the constraint $\mathbf{R}\mathbf{R}^{\dagger} = 1$ is $\mathcal{L}(\lambda, \mathbf{R}) = \lambda \langle \mathbf{U}\mathbf{U}^{\dagger} - 1 \rangle$. Where $\lambda = \langle \lambda \rangle$.

Proof of Theorem 3.3. Given the relation (3.9a) we choose the Euclidean distance to define the cost function

$$J(\mathbf{R}) = \sum_{i=1}^{\ell} \|\mathbf{B}_i - \mathbf{R} \mathbf{A}_i \mathbf{R}^{\dagger}\|^2 \quad (\text{E.8})$$

Then by *Theorem E.2* we can express the Lagrangian associated to the problem of minimizing J under the constraint that $\mathbf{R}$ is a rotator as

$$\mathcal{L}(\lambda, \mathbf{R}) = J(\mathbf{R}) + \lambda \langle \mathbf{R} \mathbf{R}^\dagger - 1 \rangle \quad (\text{E.9})$$

which by expanding we find

$$\mathcal{L}(\mathbf{R}, \lambda) = \sum_{i=1}^{\ell} |\mathbf{B}_i|^2 + |\mathbf{A}_i|^2 - 2 \langle \mathbf{B}_i^\dagger \mathbf{R} \mathbf{A}_i \mathbf{R}^\dagger \rangle + \lambda \langle \mathbf{R} \mathbf{R}^\dagger - 1 \rangle \quad (\text{E.10})$$

taking the derivative while using (B.5), (A.17d) and (A.17e) we can show that

$$\begin{aligned} \partial_{\mathbf{R}^\dagger} \mathcal{L}(\mathbf{R}, \lambda) &= \partial_{\mathbf{R}}^\dagger \mathcal{L}(\mathbf{R}) = (\partial_{\mathbf{R}} \mathcal{L}(\mathbf{R}, \lambda))^\dagger \\ &= -\frac{1}{2} \sum_{i=1}^{\ell} \left(\dot{\partial}_{\mathbf{R}} \langle \mathbf{B}_i^\dagger \dot{\mathbf{R}} \mathbf{A}_i \mathbf{R}^\dagger \rangle + \dot{\partial}_{\mathbf{R}} \langle \mathbf{B}_i^\dagger \mathbf{R} \mathbf{A}_i \dot{\mathbf{R}}^\dagger \rangle \right)^\dagger + \lambda \left(\partial_{\mathbf{R}} \langle \mathbf{R} \mathbf{R}^\dagger - 1 \rangle \right)^\dagger \\ &= -\frac{1}{2} \sum_{i=1}^{\ell} \left(\dot{\partial}_{\mathbf{R}} \dot{\mathbf{R}} * (\mathbf{A}_i \mathbf{R}^\dagger \mathbf{B}_i^\dagger) + \dot{\partial}_{\mathbf{R}} \dot{\mathbf{R}} * (\mathbf{A}_i^\dagger \mathbf{R}^\dagger \mathbf{B}_i) \right)^\dagger + \lambda \left(\partial_{\mathbf{R}} \mathbf{R} * \mathbf{R}^\dagger \right)^\dagger \\ &= -\frac{1}{2} \sum_{i=1}^{\ell} \left(\mathbf{A}_i \mathbf{R}^\dagger \mathbf{B}_i^\dagger + \mathbf{A}_i^\dagger \mathbf{R}^\dagger \mathbf{B}_i \right)^\dagger + \lambda (\mathbf{R}^\dagger)^\dagger \\ &= -\frac{1}{2} \sum_{i=1}^{\ell} \mathbf{B}_i \mathbf{R} \mathbf{A}_i^\dagger + \mathbf{B}_i^\dagger \mathbf{R} \mathbf{A}_i + \lambda \mathbf{R} \end{aligned} \quad (\text{E.11})$$

Taking $\langle \partial_{\mathbf{R}}^\dagger \rangle_{0,2} \mathcal{L} = 0$ and taking the isolated $\mathbf{R}$ to the other side of the equation we find

$$\frac{1}{2} \langle L(\mathbf{R}) \rangle_{0,2} = \lambda \mathbf{R} \quad (\text{E.12a})$$

where

$$L(\mathbf{R}) = \sum_{i=1}^{\ell} \mathbf{B}_i \mathbf{R} \mathbf{A}_i^\dagger + \mathbf{B}_i^\dagger \mathbf{R} \mathbf{A}_i. \quad (\text{E.12b})$$

Then to show that the optimal solution is when λ is the largest, we multiply both sides of $\mathbf{R}^\dagger$ and since $\mathbf{R} \mathbf{R}^\dagger = 1$ thus

$$\lambda = \frac{1}{2} L(\mathbf{R}) \mathbf{R}^\dagger \quad (\text{E.13})$$

since the left hand side is a scalar so must also be the right hand side thus we have

$$\begin{aligned} \frac{1}{2} L(\mathbf{R}) \mathbf{R}^\dagger &= \frac{1}{2} L(\mathbf{R}) * \mathbf{R}^\dagger = \frac{1}{2} \sum_{i=1}^{\ell} \langle \mathbf{B}_i^\dagger \mathbf{R} \mathbf{A}_i \mathbf{R}^\dagger + \mathbf{B}_i \mathbf{R} \mathbf{A}_i^\dagger \mathbf{R}^\dagger \rangle \\ &= \sum_{i=1}^{\ell} \langle \mathbf{B}_i^\dagger \mathbf{R} \mathbf{A}_i \mathbf{R}^\dagger \rangle \end{aligned} \quad (\text{E.14})$$

where we used (A.17b) and (A.11a) in order find that

$$\langle \mathbf{B}_i \mathbf{R} \mathbf{A}_i^\dagger \mathbf{R}^\dagger \rangle = \langle \mathbf{B}_i^\dagger (\mathbf{R} \mathbf{A}_i^\dagger \mathbf{R}^\dagger)^\dagger \rangle = \langle \mathbf{B}_i^\dagger \mathbf{R} \mathbf{A}_i \mathbf{R}^\dagger \rangle. \quad (\text{E.15})$$

Comparing the last expression of (E.14) with the cost function J , we immediately see that this is the component of J which varies with $\mathbf{R}$. Thus, to minimize J , we must maximize this quantity. Also, because of (E.13) we can readily see that the optimal solution is obtained when λ is the largest. We note that a real eigendecomposition must exist since under the product $\langle \mathbf{A} \mathbf{B}^\dagger \rangle$ (which is definite in $\mathcal{G}_3$), the multilinear function F is symmetric (see Theorem D.1), which means that a real decomposition of the matrix of F must exist. $\square$

Proof of Theorem 3.4. Let $\mathcal{Q}_{ij} = C_j(\mathbf{Q}_i)$, $\mathcal{S}_{ij} = C_j(\mathbf{S}_i)$ and let $\mathbf{S}_i = \mathbf{R}(\mathbf{P}_i)$ then considering (3.8) and (3.7a) we find that

$$C_3(\mathbf{T} \mathbf{S}_i \mathbf{T}^\dagger) = \mathbf{t} \cdot C_1(\mathbf{S}_i) + C_3(\mathbf{S}_i) = \mathbf{t} \cdot \mathcal{S}_{i1} + \mathcal{S}_{i3} \quad (\text{E.16a})$$

$$C_4(\mathbf{T} \mathbf{S}_i \mathbf{T}^\dagger) = \mathbf{t} \wedge C_1(\mathbf{S}_i) + C_4(\mathbf{S}_i) = \mathbf{t} \wedge \mathcal{S}_{i1} + \mathcal{S}_{i4} \quad (\text{E.16b})$$

Letting $\mathbf{R}$ be the identity transformation in (3.7b) we may find that

$$C_3(\mathbf{T}^\dagger \mathbf{Q}_i \mathbf{T}) = -\mathbf{t} \cdot C_1(\mathbf{Q}_i) + C_3(\mathbf{Q}_i) = -\mathbf{t} \cdot \mathcal{Q}_{i1} + \mathcal{Q}_{i3} \quad (\text{E.17a})$$

$$C_4(\mathbf{T}^\dagger \mathbf{Q}_i \mathbf{T}) = -\mathbf{t} \wedge C_1(\mathbf{Q}_i) + C_4(\mathbf{Q}_i) = -\mathbf{t} \wedge \mathcal{Q}_{i1} + \mathcal{Q}_{i4} \quad (\text{E.17b})$$

Using the defining equations (C.7) for d^2 we readily have that

$$d^2(\mathbf{T} \mathbf{S}_i \mathbf{T}^\dagger, \mathbf{Q}_i) = \|C_3(\mathbf{T} \mathbf{S}_i \mathbf{T}^\dagger) - C_3(\mathbf{Q}_i)\|^2 + \|C_4(\mathbf{T} \mathbf{S}_i \mathbf{T}^\dagger) - C_4(\mathbf{Q}_i)\|^2 \quad (\text{E.18a})$$

$$d^2(\mathbf{S}_i, \mathbf{T}^\dagger \mathbf{Q}_i \mathbf{T}) = \|C_3(\mathbf{S}_i) - C_3(\mathbf{T}^\dagger \mathbf{Q}_i \mathbf{T})\|^2 + \|C_4(\mathbf{S}_i) - C_4(\mathbf{T}^\dagger \mathbf{Q}_i \mathbf{T})\|^2 \quad (\text{E.18b})$$

Using (E.16) and (E.17) in the above equations and considering the cost function (3.10) we may write

$$\begin{aligned} J &= \|\mathbf{t} \cdot \mathcal{S}_{i1} + \mathcal{S}_{i3} - \mathcal{Q}_{i3}\|^2 + \|\mathbf{t} \wedge \mathcal{S}_{i1} + \mathcal{S}_{i4} - \mathcal{Q}_{i4}\|^2 + \\ &\quad + \|\mathbf{t} \cdot \mathcal{Q}_{i1} + \mathcal{S}_{i3} - \mathcal{Q}_{i3}\|^2 + \|\mathbf{t} \wedge \mathcal{Q}_{i1} + \mathcal{S}_{i4} - \mathcal{Q}_{i4}\|^2 \\ &= \|\mathbf{t} \cdot \mathcal{S}_{i1}\|^2 + \|\mathbf{t} \wedge \mathcal{S}_{i1}\|^2 + \|\mathbf{t} \cdot \mathcal{Q}_{i1}\|^2 + \|\mathbf{t} \wedge \mathcal{Q}_{i1}\|^2 + \\ &\quad + \|\mathcal{S}_{i3} - \mathcal{Q}_{i3}\|^2 + \|\mathcal{S}_{i4} - \mathcal{Q}_{i4}\|^2 + \|\mathcal{S}_{i3} - \mathcal{Q}_{i3}\|^2 + \|\mathcal{S}_{i4} - \mathcal{Q}_{i4}\|^2 \\ &\quad + 2\langle \mathbf{t} \cdot (\mathcal{S}_{i1} + \mathcal{Q}_{i1})(\mathcal{S}_{i3} - \mathcal{Q}_{i3})^\dagger \rangle + 2\langle \mathbf{t} \wedge (\mathcal{S}_{i1} + \mathcal{Q}_{i1})(\mathcal{S}_{i4} - \mathcal{Q}_{i4})^\dagger \rangle \end{aligned} \quad (\text{E.19})$$

Where we omitted the sum $\sum_{i=1}^N$. Recall that we are summing over the index $i = 1, 2, \dots, N$. Then using (A.30c) and (A.31) to simplify the above expression and

taking the derivative with respect to the vector $\mathbf{t}$ we have

$$\begin{aligned}
\partial_{\mathbf{t}} J &= \partial_{\mathbf{t}} \sum_{i=1}^N \mathbf{t}^2 \left(\|\mathcal{S}_{i1}\|^2 + \|\mathcal{Q}_{i1}\|^2 \right) + \\
&+ 2\partial_{\mathbf{t}} \sum_{i=1}^N \left(\mathbf{t} \cdot \langle (\mathcal{S}_{i1} + \mathcal{Q}_{i1})(\mathcal{S}_{i3} - \mathcal{Q}_{i3})^\dagger \rangle_1 + 2\mathbf{t} \cdot \langle (\mathcal{S}_{i1} + \mathcal{Q}_{i1})(\mathcal{S}_{i4} - \mathcal{Q}_{i4})^\dagger \rangle_1 \right) \\
&= 2\mathbf{t} \left(\sum_{i=1}^N \|\mathcal{S}_{i1}\|^2 + \|\mathcal{Q}_{i1}\|^2 \right) + 2 \sum_{i=1}^N \langle (\mathcal{S}_{i1} + \mathcal{Q}_{i1})(\mathcal{S}_{i4} + \mathcal{S}_{i3} - \mathcal{Q}_{i4} - \mathcal{Q}_{i3})^\dagger \rangle_1
\end{aligned} \tag{E.20}$$

which by solving for $\mathbf{t}$ we get (3.11a).

Note that, the use of (A.31) in (E.20) is valid since we are considering that $\mathbf{Q}_i$ and $\mathbf{P}_i$ are multivectors of unique grade equal to k_i . When $k_i > 1$, then the coefficients $\mathcal{S}_{i1}$ and $\mathcal{Q}_{i1}$ are of grade $k_i - 1$, $\mathcal{S}_{i3}$ and $\mathcal{Q}_{i3}$ are of grade $k_i - 2$ and $\mathcal{S}_{i4}$ and $\mathcal{Q}_{i4}$ are of grade k_i . And in the particular case when we consider $k_i = 1$ then $\mathcal{S}_{i3}$ and $\mathcal{Q}_{i3}$ are going to be equal to zero. $\square$